



Stellenbosch

UNIVERSITY
IYUNIVESITHI
UNIVERSITEIT

Autonomous Robotic Hospital Assistant

Mechanical Project 478
Final Report

Yuan Roets
25846299

Supervisor
Prof K Schreve

2025

Plagiarism declaration

I have read and understand the Stellenbosch University Policy on Plagiarism and the definitions of plagiarism and self-plagiarism contained in the Policy [Plagiarism: The use of the ideas or material of others without acknowledgement, or the re-use of one's own previously evaluated or published material without acknowledgement or indication thereof (self-plagiarism or text-recycling)].

I also understand that direct translations are plagiarism, unless accompanied by an appropriate acknowledgement of the source. I also know that verbatim copy that has not been explicitly indicated as such, is plagiarism.

I know that plagiarism is a punishable offence and may be referred to the University's Central Disciplinary Committee (CDC) who has the authority to expel me for such an offence.

I know that plagiarism is harmful for the academic environment and that it has a negative impact on any profession.

Accordingly all quotations and contributions from any source whatsoever (including the internet) have been cited fully (acknowledged); further, all verbatim copies have been expressly indicated as such (e.g. through quotation marks) and the sources are cited fully.

I declare that, except where a source has been cited, the work contained in this assignment is my own work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.

I declare that have not allowed, and will not allow, anyone to use my work (in paper, graphics, electronic, verbal or any other format) with the intention of passing it off as his/her own work.

I know that a mark of zero may be awarded to assignments with plagiarism and also that no opportunity be given to submit an improved assignment.

Signature:

Name:

Student no:

Date:

AI Use Declaration

I am convinced and can support my claim that my assessment product is an indication of my own learning, knowledge, skills, and understanding.

Where I have used AI tools for enhancing my own creation of ideas and words, I acknowledge that I have to declare it.

Where I have used AI tools for generating new ideas, words, code, image-prompts for other AI Image-generating tools, or structure and even presentations (or other AI tools that can be used as assistants to the knowledge building and representing process), I have declared and documented the use of such tools and I am prepared to talk about the process I used and what it contributed to my learning and insights.

I am aware that the lecturer can ask me to demonstrate my learning, for example through explaining the choices I made in terms of approach, content used, literature selected, conclusions drawn, etc. through an additional assessment like an oral (for example).

I understand that if I am not able to agree to the above points, there is a chance that my academic behaviour will be deemed unethical and might lead to a disciplinary case being brought against me on the grounds of cheating or plagiarism and that the standard procedures for such behaviour will be followed.

As per the Disciplinary Code of SU (par. 10.2.1 and 10.2.2), I understand that I take responsibility for the integrity of my work, which includes the obligation to ask for clarification from an academic member of staff if I am unsure of anything, and that I strictly adhered to all instructions received in the course of the academic assessment by relevant and authorized staff (whether the instruction is in oral or written format).

I understand that when I am not able to document and declare my use of AI tools, this behaviour will be deemed as cheating in examinations and assessments (Disciplinary Code 1.1 c.) as I referred to “unauthorized notes, books, electronic devices or other reference material”.

Student Number:	Signature:
Initials and Surname:	Date:

Abstract

Note that in Mechanical Project and in Mechatronic Project, this abstract is replaced by the prescribed executive summary table.

Acknowledgements

D

Table of contents

Plagiarism declaration.....	i
AI Use Declaration	ii
Abstract.....	iii
Acknowledgements	iv
Table of contents	v
List of figures.....	vii
List of tables	viii
List of symbols.....	ix
1 Introduction.....	1
1.1 Background	1
1.2 Aim and Objectives	1
1.3 Scope	2
1.4 Motivation	3
2 Literature review	4
2.1 Evolution of Autonomous Robots	4
2.2 Autonomous Robots in Healthcare Settings.....	5
2.3 The Localization Problem	5
2.4 Simultaneous Localization and Mapping	6
2.4.1 Mathematical Foundations and Problem Formulation.....	7
2.4.2 Extended Kalman Filter SLAM (EKF-SLAM)	7
2.4.3 Particle Filter SLAM and FastSLAM.....	7
2.4.4 Graph-Based SLAM	8
2.4.5 2D LIDAR SLAM for Indoor Environments.....	8
2.4.6 Occupancy Grid Mapping.....	8
2.4.7 Localization in Known Maps: AMCL	9
2.4.8 Path Planning and Navigation.....	9
2.4.9 Costmaps and Obstacle Representation	9
2.5 Robot Operating System and Modern Implementation.....	10
2.5.1 ROS Emergence: Why Robotics Needed a Common Platform.....	10
2.5.2 ROS2 Evolution: Real-time, Distributed Systems for Production Robots	10
2.5.3 Navigation Stack: How Nav2 Implements Decades of Research.....	10
2.5.4 How Modern Tools Make Complex Robotics Accessible.....	11
2.6 Current Challenges and Opportunities	12
2.6.1 Gaps in Current Systems.....	12
2.6.2 Integration Challenges	12

2.6.3	Future Directions	13
3	Technical Integration and Implementation.....	14
3.1	Hardware Design and Function	14
3.1.1	Chassis, Carriage and Electronics Compartment.....	14
3.1.2	Electronics and Circuit.....	16
3.2	Software Design and Implementation	22
3.2.1	System Architecture Overview	25
3.2.2	Hardware Abstraction and Embedded Control	28
3.2.3	Sensor Integration and Data Processing	32
3.2.4	Simultaneous Localisation and Mapping (SLAM).....	34
3.2.5	Navigation and Path Planning.....	36
3.2.6	User Interface and Navigation Control.....	40
3.2.7	System Integration and Launch Management.....	38
3.2.8	Time Synchronisation and Distributed Operation	42
3.2.9	Debugging and System Monitoring	40
3.2.10	System Limitations and Design Constraints	43
3.3	Testing and Validation	45
3.3.1	Determining the Encoder Counts.....	45
3.3.2	Motor Control Validation and PID Tuning.....	46
3.3.3	Odometry and Calibration Testing.....	47
3.3.4	Performance Testing	50
3.3.5	Failure Mode and Robustness Analysis Results	56
3.3.6	Performance Analysis and System Assessment.....	59
4	Conclusions.....	61
	References.....	62
	Appendix A	67
	Appendix B Heading for this appendix	71
B.1	Bla bla	71
B.1.1	Bla bla	71

List of figures

Figure 1 - Tank Chassis	2
Figure 2 - Shakey (Left), Stanford Cart (Middle), NavLab (Right)	4
Figure 3 - Example of a Costmap	9
Figure 4 - Electronics Compartment.....	15
Figure 5 - Storage Inside Chassis	15
Figure 6 - Carriage	16
Figure 7 – PharmaBot Wiring Diagram.....	22
Figure 8 - Software Architecture for PharmaBot.....	23
Figure 9 - Nodes and Topics During Runtime.....	26
Figure 10 - Serial Motor Bridge Command Frame.....	26
Figure 11 - Transform Tree	32
Figure 12 - User Interface	42
Figure 13 - Tuned Response to Step Input	47
Figure 14 - Lipo Battery	68
Figure 15 - Okdo LiDAR Scanner	68
Figure 16 - Motor.....	68
Figure 17 - Arduino Nano.....	68
Figure 18 - TB6612FNG Motor Driver	68
Figure 19 - Raspberry Pi Model 4B.....	68
Figure 20 - Waveshare Servo Driver Hat	69

List of tables

Table 1: System power consumption across operational modes:	17
Table 2 - Encoder Values for 10 Rotations:	45
Table 3 - Linear Distance Calibration Data	48
Table 4 - Rotational Calibration Data	49
Table 5 - Odometry Drift Test Results	50
Table 6 - Single Point Navigation Performance:	52
Table 7 - Return-to-Home Single Waypoint Performance:	53
Table 8 - Sequential Waypoint Navigation Performance:	54
Table 9 - Obstacle Avoidance Test Results:	55
Table 10 - Low-Height Obstacle Detection Results:	56
Table 11 - Surface Condition Performance Assessment:	56
Table 12 - Trapped/Stuck Scenario Results:.....	57
Table 13 - Communication Failure Response:	58
Table 14 - Node Failure Recovery Performance:	59
Table 15 - Overall System Reliability Metrics:	60

List of symbols

σ	Standard deviation
$\pm$	Plus-minus (e.g., mean $\pm \sigma$)
$^{\circ}$	Degrees (angular measurement)
mm	Millimetres
m	Meters
s	Seconds
%	Percent
CW	Clockwise (used in angular error)
N/A	Not applicable
EKF	Extended Kalman Filter
SLAM	Simultaneous Localization and Mapping
AMCL	Adaptive Monte Carlo Localization
Nav2	Navigation Stack in ROS2
ROS2	Robot Operating System 2
UART	Universal Asynchronous Receiver-Transmitter
DDS	Data Distribution Service
PID	Proportional-Integral-Derivative Controller
URDF	Unified Robot Description Format
EMI	Electromagnetic Interference
mAh	Milliampere-hour (battery capacity)
V	Volt (voltage)

1 Introduction

1.1 Background

The integration of robotics in healthcare is transforming hospital operations by improving efficiency, reducing staff workload, and streamlining logistical tasks. With hospitals facing increasing patient loads and administrative duties, there is a growing need for automation to optimize workflow and reduce non-clinical responsibilities for medical personnel. One promising application is the use of autonomous robots for the transportation of supplies, documents, and specimens between departments.

This project is part of a competition, and specific rules and constraints apply, e.g. budget and resource constraints, a standardized platform, laboratory facilities, and procurement processes. The project has been provided with preselected components, which can be used for development.

The robot will be floor-bound, capable of obstacle avoidance, and will follow predefined routes within the hospital environment. It will feature a storage compartment where staff can place items for transport, but the robot itself will not handle loading or unloading. Instead, it will rely on hospital staff to complete these tasks while it autonomously moves between designated points.

Many existing robotic systems employ Simultaneous Localization and Mapping (SLAM) to dynamically map and navigate their environment. This project will make use of Robot Operating System (ROS2) and employ its Navigation (Nav2) stack to perform SLAM.

1.2 Aim and Objectives

The primary aim of this project is to design, build, and test an autonomous hospital-based delivery and collection robot. The specific objectives include:

1. Design a floor-bound hospital robot capable of navigating autonomously along predefined routes.
2. Implement an interface for hospital staff to send collection and delivery requests.
3. Test the robot's performance under controlled conditions and within the framework of the competition.

The focus of the project is academic, evaluating the technical and operational aspects rather than competing with commercially available hospital robots. While the project will explore the potential long-term impact of hospital-based

autonomous robots, the emphasis remains on engineering feasibility and meeting predefined performance metrics.

1.3 Scope

The robot must operate within the rules and constraints set by the competition. The development process will be limited to using the provided components, including the tank chassis (Figure 1) and will focus on implementing a functional system within these constraints. A fixed budget is available to buy any additional components.



Figure 1 - Tank Chassis

The initial components include:

- 1 x Treaded platform incl. 2 x geared DC motor and motor driver board (Must be used)
- 1 x LIDAR
- 1 x Raspberry Pi 4 Model B
- 1 x Micro HDMI cable
- 1 x Micro SD Card with adapter
- 1 x Battery, 8000 mAh, 11.1 V
- 1 x Raspberry Pi PSU
- 2 x Battery connectors
- 1 x USB-C data cable

The key areas of development include the following.

- Navigation and Autonomy: The robot will follow predefined routes, employing SLAM for localization and mapping to further perform obstacle avoidance.

- **Task Execution:** The robot will perform on-demand collection and delivery of items, activated through an interface-based command system.
- **User Interaction:** Hospital staff will communicate with the robot via the designated interface, specifying pickup and drop-off locations.
- **Safety and Reliability:** The system must operate safely in a hospital setting, avoiding obstacles and ensuring efficient operation without interfering with hospital workflows.

The robot is not required to:

- Physically load or unload items, it will rely on hospital staff for this function.
- Navigate complex environments beyond predefined routes.

1.4 Motivation

The increasing demand for hospital efficiency, combined with the administrative burden on healthcare professionals, highlights the need for autonomous solutions. Nurses and hospital staff often spend valuable time transporting medical supplies, laboratory samples, and documents across hospital wards, which can be time-consuming and inefficient. By automating these tasks, hospital staff can focus on patient care, improving overall workflow and reducing human error.

Furthermore, hospital environments are structured yet dynamic, making them an ideal testing ground for robotic solutions. The ability to remotely control a robot with something such as a chatbot presents an intuitive and user-friendly approach, reducing the learning curve for hospital staff. Additionally, evaluating navigation strategies, such as SLAM, provides valuable insights into the practicality of specific robotic approaches in healthcare environments.

This study will serve as a technical feasibility evaluation, with findings that could inform future improvements and adaptations.

2 Literature review

2.1 Evolution of Autonomous Robots

The journey toward truly autonomous robots began in 1966 with Shakey, developed at Stanford Research Institute. This refrigerator-sized robot, equipped with a television camera, triangulating rangefinder, and bump sensors, represented humanity's first serious attempt at creating a machine that could perceive, reason, and act independently in an unstructured environment. Shakey could navigate simple room layouts, push blocks, and even plan multi-step tasks, capabilities that seemed almost magical at the time. However, this early pioneer was painfully slow, requiring up to an hour to analyse a scene and plan a simple manoeuvre, while being tethered to a room-sized computer for processing power (Nilsson, 1984).

The limitations of Shakey sparked decades of innovation. By the 1980s, robots like the Stanford Cart had demonstrated autonomous navigation across outdoor terrain using stereo vision (Moravec, 1983), while Carnegie Mellon's NavLab project placed autonomous systems in full-sized vehicles, achieving highway speeds by the 1990s (Thorpe et al., 1988). These advances were driven by exponential improvements in computational power, sensor miniaturization, and the development of probabilistic approaches to handling uncertainty, the fundamental challenge that had plagued early autonomous systems.

The transformation from Shakey's hour-long deliberations to today's real-time autonomous systems represents one of robotics' greatest achievements. Modern autonomous robots can simultaneously build maps of unknown environments, localize themselves within those maps, and navigate safely around dynamic obstacles, all while making hundreds of decisions per second. This evolution has opened possibilities that seemed like science fiction just decades ago: autonomous vehicles navigating busy streets, drones delivering packages, and robots working alongside humans in factories and offices (Raj and Kos, 2022).

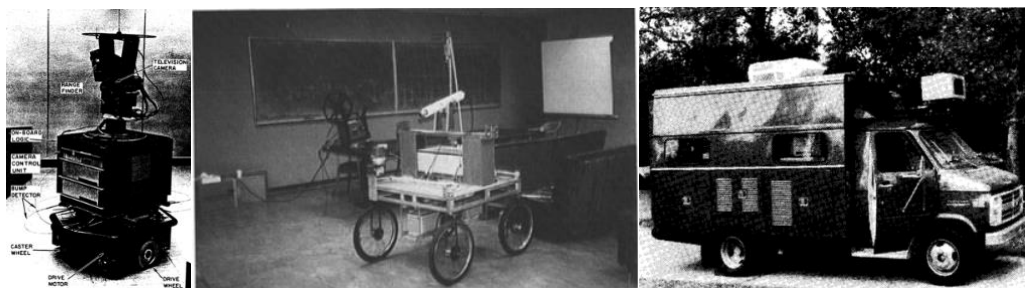


Figure 2 - Shakey (Left), Stanford Cart (Middle), NavLab (Right)

2.2 Autonomous Robots in Healthcare Settings

The healthcare industry's adoption of autonomous robots began in earnest during the 1990s, driven by mounting pressure to improve efficiency while reducing costs. Time-motion studies have consistently demonstrated that nurses and hospital staff spend significant portions of their shifts on non-clinical tasks, including transporting medications, laboratory samples, and medical supplies between departments. Yen et al. (2018) found that nurses spent approximately 10% of their time on delegable and non-nursing activities that could be performed by other personnel, representing a concerning diversion of skilled healthcare workers from direct patient care.

The first successful autonomous hospital delivery robot, HelpMate, was deployed at Danbury Hospital in Connecticut in 1991 (Evans, 1990). This pioneering system used a combination of ultrasonic ranging sensors, structured light vision, and pre-programmed topographical knowledge bases to navigate hospital corridors, delivering medications, laboratory specimens, and supplies to nursing stations. Evans (1990) reported that over 50 HelpMate units were operational by the mid-1990s, working up to 24 hours per day in uncontrolled hospital environments. Despite its primitive navigation capabilities by today's standards, HelpMate demonstrated the viability of autonomous robots in healthcare environments and established the foundation for modern hospital robotics.

Modern hospital robots have evolved significantly beyond HelpMate's basic point-to-point navigation capabilities. Contemporary systems such as the TUG robot, developed by Aethon, employ advanced sensor fusion combining multiple modalities including LiDAR, cameras, and ultrasonic sensors to navigate complex hospital environments (Melanson, 2015). These robots utilize simultaneous localization and mapping (SLAM) technology to operate adaptively in previously unmapped areas and handle dynamic obstacles (Fang et al., 2021).

However, significant challenges persist in hospital robotics deployment. Hospital environments present unique obstacles for autonomous navigation: narrow corridors frequently occupied by mobile equipment, unpredictable human traffic patterns, strict hygiene and infection control requirements, and the critical need for absolute reliability in life-or-death situations. Unlike industrial settings where robots operate in controlled environments with predictable workflows, hospital robots must seamlessly integrate into spaces designed entirely for human use while maintaining safety standards and operational efficiency. These constraints make robust navigation algorithms, sophisticated human-robot interaction protocols, and fail-safe operational procedures paramount to successful deployment.

2.3 The Localization Problem

The fundamental challenge underlying all autonomous navigation is deceptively simple: "Where am I?" This question, known as the localization problem, has driven

decades of robotics research and represents the cornerstone upon which all autonomous behaviour depends. Without accurate knowledge of its position and orientation, an autonomous robot cannot plan paths, avoid obstacles, or complete tasks reliably.

Early mobile robots attempted to solve localization through dead reckoning, estimating position by integrating wheel encoder measurements over time. While conceptually straightforward, this approach suffered from cumulative errors that rendered it useless for long-term autonomous operation. Small inaccuracies in wheel slip, mechanical tolerances, and sensor drift would compound over time, causing robots to become lost even in familiar environments (Park et al., 1996).

The breakthrough came with the development of landmark-based localization systems in the 1980s. By identifying distinctive features in the environment—such as walls, doorways, or artificial beacons, robots could correct their position estimates and prevent the accumulation of dead reckoning errors. However, these systems required either modification of the environment (installing beacons) or relied on the consistent presence of recognizable landmarks, limiting their applicability in dynamic settings like hospitals where equipment and layouts frequently change.

The evolution toward probabilistic localization methods in the 1990s marked a paradigm shift in autonomous robotics (Raj and Kos, 2022). Rather than attempting to maintain a single, precise position estimate, probabilistic approaches represent the robot's belief about its location as a probability distribution over all possible positions. This framework naturally handles uncertainty and ambiguity, allowing robots to operate effectively even when sensor data is noisy or incomplete (Thrun et al., 2005).

The Monte Carlo Localization (MCL) algorithm, also known as particle filter localization, emerged as the dominant approach for mobile robot localization. By maintaining thousands of hypotheses about the robot's possible locations and continuously updating these hypotheses based on sensor observations, MCL can recover from localization failures and operate robustly in dynamic environments, exactly the capabilities needed for hospital robotics applications (Dellaert et al., 1999).

Yet even sophisticated localization algorithms depend on one critical assumption, the existence of an accurate map of the environment. This requirement led researchers to the next frontier, what if robots could build their own maps while simultaneously localizing within them?

2.4 Simultaneous Localization and Mapping

The simultaneous localization and mapping (SLAM) problem represents one of the most significant achievements in autonomous robotics, addressing the fundamental

chicken-and-egg dilemma: to localize accurately, a robot needs a map of its environment, but to build a map, it must know where it is (Raj and Kos, 2022). SLAM solves this by estimating both the robot's trajectory and the map of the environment concurrently, using only the robot's sensor observations and control inputs. (Gibbons et al., 2000)

2.4.1 Mathematical Foundations and Problem Formulation

The SLAM problem can be formally expressed as estimating the posterior probability distribution over the robot's trajectory $x_{0:t}$ and the map m , given the sequence of sensor observations $z_{1:t}$ and control inputs $u_{1:t}$:

$$P(x_{0:t}, m \mid z_{1:t}, u_{1:t}) \quad (1)$$

This joint posterior captures the inherent uncertainty in both the robot's pose and the environmental map. The sequential nature of the problem allows for recursive Bayesian estimation, where the current state estimate depends only on the previous state and the most recent observation (Thrun et al., 2005).

2.4.2 Extended Kalman Filter SLAM (EKF-SLAM)

The Extended Kalman Filter provided the first practical solution to the SLAM problem by maintaining a Gaussian approximation of the joint posterior. EKF-SLAM represents the state vector as a combination of the robot pose and landmark positions:

$$x = [x_r, y_r, \theta_r, x_1, y_1, \dots, x_n, y_n]^T \quad (2)$$

where (x_r, y_r, θ_r) represents the robot position (pose) and (x_i, y_i) represents the positions of n landmarks. The algorithm alternates between prediction and update steps, using the robot's motion model for prediction and sensor observations for correction (Solà et al., 2025).

However, EKF-SLAM suffers from several limitations. The computational complexity scales quadratically with the number of landmarks (Cadena et al., 2008), making it impractical for large environments. More critically, the Gaussian assumption fails when the robot faces ambiguous situations or when linearization errors accumulate, leading to inconsistent estimates and potential divergence (Bailey and Durrant-Whyte, 2006).

2.4.3 Particle Filter SLAM and FastSLAM

The FastSLAM algorithm addressed EKF-SLAM's limitations by factoring the joint posterior using the insight that landmark positions are conditionally independent given the robot's trajectory:

$$P(x_{0:t}, m \mid z_{1:t}, u_{1:t}) = P(x_{0:t} \mid z_{1:t}, u_{1:t}) \prod_i P(m_i \mid x_{0:t}, z_{1:t}, u_{1:t}) \quad (3)$$

This factorization allows FastSLAM to use a particle filter for robot trajectory estimation while maintaining separate EKFs for each landmark conditioned on each particle's trajectory. The result is $O(M \log N)$ complexity, where M is the number of particles and N is the number of landmarks, making large-scale SLAM computationally feasible (Montemerlo et al., 2002).

2.4.4 Graph-Based SLAM

Modern SLAM systems increasingly employ graph-based approaches, where robot poses and landmarks are represented as nodes in a graph, connected by edges representing spatial constraints from sensor observations. The SLAM problem becomes one of finding the configuration of nodes that best satisfies all constraints, typically solved through nonlinear optimization techniques such as the Gauss-Newton or Levenberg-Marquardt algorithms.

Graph-based SLAM excels at handling loop closures—situations where the robot revisits previously mapped areas. When a loop closure is detected, the accumulated drift in the robot's trajectory can be corrected by optimizing the entire graph structure, resulting in globally consistent maps. (Grisetti et al., 2010)

2.4.5 2D LIDAR SLAM for Indoor Environments

For indoor applications like hospital robotics, 2D LIDAR sensors provide an ideal balance of accuracy, computational efficiency, and cost-effectiveness. LIDAR SLAM algorithms typically employ scan matching techniques to align consecutive laser scans, estimating the robot's motion between scans. The Iterative Closest Point (ICP) algorithm and its variants are commonly used for this purpose, minimizing the distance between corresponding points in successive scans (Jiang et al., 2022).

The GMapping algorithm, based on particle filter SLAM, has become a standard for 2D LIDAR SLAM in indoor environments. It uses a proposal distribution that incorporates both the motion model and the most recent laser scan, improving particle efficiency and reducing the required number of particles. (Lin et al., 2017)

2.4.6 Occupancy Grid Mapping

The output of 2D LIDAR SLAM is typically an occupancy grid map, where the environment is discretized into a grid of cells, each containing a probability of occupancy. The occupancy probability for cell c given sensor measurements $z_{1:t}$ is updated using the log-odds representation:

$$l(c | z_{1:t}) = l(c | z_{1:t-1}) + \log(P(c | z_t)) - \log(P(c)) \quad (4)$$

Where l represents the log-odds value. This approach efficiently handles the uncertainty inherent in sensor measurements while providing a format suitable for path planning algorithms (Thrun et al., 2005).

2.4.7 Localization in Known Maps: AMCL

Once a map is available, the robot can perform localization using the Adaptive Monte Carlo Localization (AMCL) algorithm. AMCL maintains a particle filter representation of the robot's pose belief, dynamically adjusting the number of particles based on the localization uncertainty. The algorithm uses a motion model to predict particle positions and a sensor model to weight particles based on their likelihood given the current sensor observations (Jiang et al., 2022).

The key advantage of AMCL is its ability to recover from localization failures. When the robot becomes lost, the particle distribution spreads across the map, and the algorithm can relocalize by matching sensor observations to the known map structure (Fox, 2001).

2.4.8 Path Planning and Navigation

With localization and mapping capabilities established, autonomous navigation requires sophisticated path planning algorithms. The navigation stack typically employs a two-layer approach: global path planning for long-term route planning and local path planning for immediate obstacle avoidance.

Global planners, such as Dijkstra's algorithm or A*, operate on the occupancy grid map to find optimal paths from the robot's current position to the goal. These algorithms account for static obstacles represented in the map but cannot handle dynamic obstacles encountered during navigation. (Raj and Kos, 2022)

Local planners address dynamic obstacle avoidance through algorithms like the Dynamic Window Approach (DWA), which selects control commands by optimizing a multi-objective function considering path progress, obstacle clearance, and velocity constraints. The DWA algorithm evaluates candidate trajectories within the robot's dynamic window—the set of velocities reachable given the robot's current velocity and acceleration limits. (Jiang et al., 2022)

2.4.9 Costmaps and Obstacle Representation

Modern navigation systems use layered costmaps to represent different types of obstacles and constraints. The static layer represents permanent obstacles from the map, while the obstacle layer handles dynamic obstacles detected by sensors. Inflation layers expand obstacle boundaries to account for the robot's size and provide safety margins (Jiang et al., 2022).

The costmap assigns numerical costs to each cell, where high costs represent areas to avoid and low costs indicate preferred paths. Path planning algorithms incorporate these costs to generate trajectories that balance efficiency with safety.

Insert Figure here

Figure 3 - Example of a Costmap

2.5 Robot Operating System and Modern Implementation

2.5.1 ROS Emergence: Why Robotics Needed a Common Platform

The Robot Operating System (ROS) emerged from the recognition that robotics research was being severely hampered by the lack of standardized software infrastructure. Prior to ROS, each research group and robotics company developed proprietary software stacks, leading to significant duplication of effort and limited code reuse across the robotics community. Since ROS was started in 2007, a lot has changed in the robotics and ROS community, transforming from a research tool into the de facto standard for robotics software development.

The original ROS provided essential services that every robotics application required: hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management. However, as robotics moved from laboratory environments to real-world applications, fundamental limitations became apparent. ROS 1 was designed primarily for research scenarios with single-robot systems operating in controlled environments, lacking the real-time capabilities, security features, and distributed system support required for production robotics applications (Renard, 2024).

2.5.2 ROS2 Evolution: Real-time, Distributed Systems for Production Robots

To address these weaknesses, ROS underwent a major update and the first version of ROS2 was released back in 2015, being lingeringly adopted by the community in recent years. The transition to ROS2 represented a fundamental architectural redesign rather than an incremental improvement. The new system was built from the ground up to address the critical limitations of ROS 1, with particular focus on real-time performance, security, and multi-robot systems (The Construct, 2019).

The most significant architectural change in ROS2 was the adoption of Data Distribution Service (DDS) as the underlying middleware. This provided several key advantages: quality of service (QoS) policies for reliable communication, built-in security features, and native support for distributed systems. ROS2 includes authentication and other security improvements that make it suitable for production environments where safety and reliability are paramount (ROS2 Design, 2019).

ROS2's real-time capabilities stem from its deterministic communication patterns and support for real-time operating systems. This is crucial for applications like autonomous vehicles and industrial robotics where precise timing is essential.

2.5.3 Navigation Stack: How Nav2 Implements Decades of Research

The Navigation2 (Nav2) stack represents the culmination of decades of robotics research, packaged into a production-ready software framework. Nav2 uses

behaviour trees to create customized and intelligent navigation behaviour by orchestrating many independent modular servers. Behaviour trees are hierarchical decision-making structures that enable robots to select and execute actions based on environmental conditions and task requirements. Unlike traditional finite state machines, behaviour trees provide a modular and composable framework where complex behaviours emerge from simple, reusable building blocks organized in a tree structure with nodes representing conditions, actions, and control flow logic (Macenski et al., 2025). This architecture allows for complex decision-making processes that can handle the uncertainty and dynamic nature of real-world environments.

The Nav2 stack integrates the SLAM algorithms, localization methods, and path planning techniques discussed in previous sections into a cohesive system. The ROS2 Navigation stack is a collection of software packages that help a mobile robot move from a starting location to a goal location safely. The modular design allows developers to swap components based on their specific requirements, such as different SLAM algorithms or path planners (Nav2 Team, 2025).

Key components of the Nav2 stack include the planner server (for global path planning), controller server (for local trajectory tracking), behaviour tree navigator (for high-level decision making), and recoveries server (for handling failure cases). ROS2 action servers are used to communicate with BT navigator through a NavigateToPose action message to request for navigation, providing a standardized interface for navigation commands (Horicam et al., 2022).

2.5.4 How Modern Tools Make Complex Robotics Accessible

The integration of modern simulation tools, visualization software, and development environments has dramatically lowered the barrier to entry for robotics development. Gazebo simulation environment allows for realistic testing of robotic systems before deployment, while RViz provides powerful visualization capabilities for debugging and monitoring robot behaviour (Renard, 2024). The ROS2 navigation stack using the slam_toolbox for mapping an environment and the core Nav2 packages to navigate in the mapped environment (Nav2 Team, 2025).

The `ros2_control` framework provides standardized interfaces for hardware abstraction, making it possible to develop control algorithms that work across different robotic platforms (Addison, 2024). This is particularly important for hospital robotics applications where different manufacturers' robots may need to use the same navigation and control software.

Modern development workflows typically involve simulation-first development, where algorithms are developed and tested in simulation before being deployed to physical hardware. This approach significantly reduces development time and allows for more comprehensive testing of edge cases and failure scenarios.

2.6 Current Challenges and Opportunities

2.6.1 Gaps in Current Systems

Despite the maturation of individual components like SLAM algorithms and path planning, several critical gaps persist in current autonomous hospital robotics systems. A significant challenge lies in the robust integration of multiple sensor modalities for perception in dynamic hospital environments (Wevolver, 2025). While LIDAR-based navigation is reliable, it can struggle with certain materials and lighting conditions common in hospitals, necessitating diverse sensor inputs for comprehensive environmental understanding. Multi-sensor fusion technologies have emerged as critical solutions for achieving high-precision localization in mobile robots operating within dynamic and unstructured environments, as sensor frequency asynchrony, drift accumulation, and measurement noise remain persistent challenges (Huang et al., 2025).

Current systems also contend with complexities in human-robot interaction (HRI) within healthcare settings. Hospital staff possess varied technical expertise, demanding intuitive interfaces that do not require specialized robotics knowledge (Boston Engineering, 2024). Furthermore, the standardization of task specification and communication protocols for seamless integration with existing hospital management systems remains an ongoing challenge. Research on autonomous hospital transport systems has highlighted the need for novel frameworks combining high-level task control with optimized multi-modal user interfaces, particularly for elderly persons and emergency detection during transportation (Zachariae et al., 2024).

Power management and ensuring long-term autonomous operation are additional hurdles. Hospital robots require continuous operation for extended periods while maintaining precise navigation. This necessitates further research into efficient battery management, optimized charging strategies, and graceful degradation of capabilities as power levels decline (Murphy, 2019).

2.6.2 Integration Challenges

The integration of diverse hardware components, including sensors, actuators, computer platforms, and communication systems, presents significant engineering challenges. Real-time coordination among perception, planning, and control systems requires careful consideration of computational resources and timing constraints. Modern hospital robots must strike a balance between computational complexity, power consumption, and heat generation (Siciliano and Khatib, 2008).

Human-robot interaction in hospitals differs significantly from industrial or domestic settings. Healthcare workers often operate under time pressure and stress, requiring robot interfaces that are not only intuitive but also fail-safe. Robots must also adapt to the nuanced social dynamics of healthcare, such as prioritizing

emergency situations and interpreting implicit communication cues from staff (Jones et al., 2019).

Safety certification for medical environments introduces another layer of complexity. Unlike industrial robots operating in controlled environments with trained personnel, hospital robots must safely interact with a diverse population, including patients, visitors, and staff with varying levels of mobility and awareness (Occupational Safety and Health Administration, 2023). This necessitates rigorous safety protocols and compliance with medical device regulations.

2.6.3 Future Directions

The future of autonomous hospital robotics points towards deeper integration with hospital information systems, enabling robots to access and utilize patient records, medication databases, and scheduling systems. Machine learning and artificial intelligence will play increasingly pivotal roles in enabling robots to adapt their behaviour to specific hospital environments and dynamic workflow patterns (Jones et al., 2019).

Multi-robot coordination is an emerging area of research, offering the potential for teams of robots to collaboratively handle complex logistics tasks. This includes capabilities such as load balancing, redundancy for critical deliveries, and coordinated responses to emergency scenarios (Murphy, 2020).

The incorporation of advanced sensing technologies, including sophisticated computer vision, tactile sensing, and environmental monitoring, will empower robots to better understand and proactively respond to their surroundings. This could involve detecting spills, monitoring air quality, or identifying maintenance needs within the hospital infrastructure (Shashikala and Sharma, 2022).

Edge computing and 5G connectivity are poised to enable more sophisticated distributed intelligence. This will allow robots to offload computationally intensive tasks to cloud resources while maintaining real-time responsiveness for critical on-board functions (Lin et al., 2020).

3 Technical Integration and Implementation

The complexity of this project did not lie only in the hardware and software development, but in creating a way of allowing so many complex systems to interface correctly and efficiently. Modern autonomous robotics systems require careful consideration of hardware-software integration, power management, thermal design, and modularity to ensure reliable operation in demanding environments. The aim of this chapter is to explain the hardware and software design and design choices, as well as showcasing how emergent problems were mitigated by ensuring the system works well together. The systems engineering approach adopted prioritizes modularity, scalability, and maintainability while meeting the stringent requirements of autonomous hospital navigation.

3.1 Hardware Design and Function

The hardware design focuses on robust mechanics, precise sensor integration, and efficient power management to support autonomous operation in a hospital setting. The design philosophy emphasizes modularity to facilitate future upgrades, ease of maintenance, and system reliability. Each component was selected through a systematic evaluation process considering performance specifications, power consumption, cost, and integration complexity. The complete hardware integration is illustrated in the wiring diagram (Figure 7).

3.1.1 Chassis, Carriage and Electronics Compartment

The mechanical design represents a critical foundation for autonomous operation, with careful attention to weight distribution, accessibility, modularity, and protection of sensitive electronics. It should be noted that hardware design was not the primary focus of this project. Design decisions, including the use of 3D printing and material selection, were primarily driven by rapid prototyping considerations rather than production optimization.

3.1.1.1 Chassis

The provided Hiwonder tank chassis (Figure 1) serves as the foundation platform for the autonomous hospital robot. The tank track system offers two main advantages: traction, due to the large amount of ground contact, and manoeuvrability, since the tracks can spin in opposite directions, allowing the robot to turn around while remaining in one place.

The chassis features an integrated suspension system that provides mechanical filtering of vibrations, protecting sensitive electronics and maintaining stable sensor readings during operation. This is particularly crucial for LIDAR accuracy, where mechanical vibrations can introduce noise into range measurements. The chassis is

also strong and lightweight due to its aluminium construction. Multiple mounting points throughout the chassis allow for modular expansion (Figure 1).

All critical electronics (the Raspberry Pi, Arduino Nano, and motor driver) were placed within the chassis, providing inherent protection against tampering and accidental damage. The chassis has a semi-open construction, allowing good airflow over the components for adequate convective cooling as the robot moves.

3.1.1.2 Electronics Compartment



Figure 4 - Electronics Compartment



Figure 5 - Storage Inside Chassis

The custom-designed electronics compartment (Figure 4) addresses several critical requirements: secure battery mounting, easy maintenance access, circuit protection, and systematic power distribution. The compartment was manufactured using fused deposition modelling (FDM) with PLA filament. FDM was selected primarily for its rapid and cost-effective prototyping capabilities while providing adequate mechanical properties for the static loads encountered. Storage inside the chassis is shown in Figure 5.

The battery compartment is accessible by removing two small pins on the side of the compartment and then removing the lid. This design reflects the operational requirement that LiPo batteries should be removed for charging to ensure safety and battery longevity. The compartment also houses a 6A blade fuse in an accessible holder.

The terminal strip integration provides a centralized distribution point for the 11.1V supply rail, facilitating systematic circuit troubleshooting and expansion. This

modular approach to power distribution simplifies maintenance and modification procedures.

3.1.1.3 Carriage

[Placeholder for image]

Figure 6 - Carriage

The cargo carriage (Figure 6) represents the functional interface between the robotic platform and the hospital logistics requirements. The design dimensions were optimized to accommodate standard hospital items including medication containers, prescriptions, and small medical supplies while maintaining the robot's stability margins.

The integrated LiDAR bridge elevates the sensor 260 mm above the carriage surface, providing unobstructed 360° scanning capability. The rear cutout design facilitates rapid loading and unloading by hospital staff, providing easy access for item retrieval while maintaining a small lip to prevent cargo displacement during acceleration.

The decision to employ 3D printing was driven by rapid prototyping requirements for this research project. While sheet metal construction would provide superior impact resistance and lower material costs for volume production, the prototyping advantages of additive manufacturing were decisive.

3.1.2 Electronics and Circuit

The electronic system design follows a distributed architecture approach, partitioning computational tasks across multiple processors to optimize performance, power consumption, and system reliability.

3.1.2.1 Power Management and Battery Life Analysis

Power Distribution Architecture: The system utilizes a Gens Bashing 3S LiPo battery (11.1V nominal, 8000mAh, Figure 1), with hierarchical voltage regulation. A 6A fuse provides overcurrent protection based on worst-case analysis, while a terminal strip implements distribution minimizing voltage drops.

Table 1: System power consumption across operational modes:¹

Component	Idle (W)	Cruise (W)	Peak (W)
Raspberry Pi 4	4.44	6.66	8.88
LIDAR LD19	2.00	2.00	2.00
Arduino Nano	0.44	0.44	0.44
Motor Drivers	0.22	0.89	1.11
Motors	0.00	6.66	22.20
Total	7.10	16.65	34.63

Idle operation time:

$$t_{idle} = \frac{C_{battery} \times DoD}{I_{idle}} = \frac{8000 \text{ mAh} \times 0.8}{640 \text{ mA}} = 10.0 \text{ hours} \quad (5)$$

Cruise operation time:

$$t_{cruise} = \frac{8000 \text{ mAh} \times 0.8}{1500 \text{ mA}} = 4.27 \text{ hours} \quad (6)$$

Mixed operation (60% cruise, 20% idle, 10% peak, 10% charging):

$$I_{avg} = 0.6 \times 1.5 + 0.2 \times 0.64 + 0.1 \times 3.1 = 1.34 \text{ A} \quad (7)$$

$$t_{mixed} = \frac{6400 \text{ mAh}}{1340 \text{ mA}} = 4.78 \text{ hours} \quad (8)$$

3.1.2.2 Waveshare servo motor driver

The Waveshare Servo Driver HAT (Figure 26) was selected to address two critical system limitations: the Raspberry Pi's limited PWM channels (only 2 hardware PWM outputs) and insufficient GPIO current capability (16mA per pin, 50mA total). The HAT's PCA9685 chip provides 16 independent PWM channels, enabling motor control while providing capacity for future expansion including robotic arms, security roller covers, or additional LIDAR scanners.

Furthermore, the 12-bit resolution significantly exceeds the Pi's native 10-bit PWM:

¹ Battery Life Calculations: Assuming 80% depth of discharge (DoD).

$$\text{PWM resolution improvement} = \frac{2^{12}}{2^{10}} = \frac{4096}{1024} = 4 \times \text{better granularity} \quad (9)$$

An important secondary benefit is the HAT's integrated buck regulator, which enables direct LiPo-to-Pi power conversion. The LiPo connects to the HAT's VIN terminal, and the HAT powers the Pi via the 40-pin connector, reducing system complexity and minimizing form factor.

The HAT exposes multiple 5V and GND pins, addressing the Pi's current limitations and enabling clean power distribution to sensors and peripherals without overloading individual GPIO pins. This modular approach supports system expansion without circuit redesign.

3.1.2.3 Raspberry Pi Model 4B

The Raspberry Pi Model 4B (Figure 25) acts as the main onboard computer, handling real-time sensor processing and low-level navigation control. It was selected over alternatives like the NVIDIA Jetson Nano or Intel NUC due to its balance between performance, power efficiency, cost, and software support ecosystem.

The quad-core ARM Cortex-A72 processor, running at 1.5 GHz, manages tasks such as processing LiDAR data at 10 Hz with acceptable latencies in ROS2 nodes. The 8 GB RAM version supports running multiple concurrent ROS2 processes and storing mapping data. Computationally intensive tasks such as SLAM and path planning are offloaded to a remote development machine, while local control remains on the Pi. This distributed architecture improves scalability, eases development, reduces power consumption, and allows easier upgrades.

While a Jetson might offer superior performance for advanced AI tasks, the cost differential was not justified for this application. Using only Arduino microcontrollers was not feasible, as they lack the processing power and multitasking capabilities required for complex software frameworks like ROS2 and sensor fusion.

Key connections include UART for the LiDAR and USB for the Arduino Nano. A passive heatsink and fan maintain operating temperatures around 65°C under load to prevent thermal throttling.

3.1.2.4 Arduino Nano

The Arduino Nano (Figure 23) manages real-time motor control and encoder readings, tasks requiring deterministic timing that the Raspberry Pi's non-real-time operating system cannot consistently guarantee, especially under heavy computational loads. The ATmega328P microcontroller handles interrupts precisely for encoder tracking and PID control.

The Arduino Nano was selected over alternatives like the ESP32 or STM32 for its low cost, simplicity, and sufficient performance (16 MHz for 50 Hz PID updates). Power consumption is approximately 20 mA, contributing minimally to overall system power draw.

The implementation uses hardware interrupts on pins 2 and 3 for encoder resolution below 0.5% of wheel rotation, runs PID loops at 50 Hz for speed control, and communicates via UART serial with the Raspberry Pi to transmit odometry data and receive velocity commands.

3.1.2.5 Motor Driver (DFRobot TB6612FNG)

The DFRobot TB6612FNG motor driver (Figure 24) was selected over alternatives like the L298N or DRV8833 for its superior efficiency, current capacity, compact size, and straightforward control interface.

The MOSFET H-bridge architecture achieves approximately 95% efficiency, significantly reducing heat generation. At a typical operating current of 800 mA with an on-resistance of 0.5 ohms per channel, the power loss is:

$$P_{loss} = I^2 R = (0.8)^2 \times 0.5 = 0.32 \text{ W per channel.} \quad (10)$$

The driver handles 1.2 A continuous (3.2 A peak), matching the motors' 1.5 A stall current specification. Control requires only two pins per motor (PWM and direction), conserving Arduino GPIO resources.

Connections include PWM and direction signals from the Arduino Nano, 11.1 V from the LiPo for motor voltage, and 5 V logic compatible with the Arduino. Built-in protections cover overcurrent, thermal shutdown at 175°C, and undervoltage conditions.

3.1.2.6 Motors and Encoders

The JGB3865-520R45-12 geared DC motors (Figure 22) suit mobile robots, with a 45:1 ratio, 12 V operation, 1.5 A stall current, and 0.5 N·m stall torque.

The maximum linear velocity of the robot, based on the wheel diameter D of 0.065 m and a no-load motor speed of 150 RPM after gearing, is given by

$$v_{\max} = \frac{\pi \times D \times \text{RPM}}{60} = \frac{\pi \times 0.065 \times 150}{60} = 0.51 \text{ m/s.} \quad (11)$$

The output torque at the wheel, assuming 85% gearbox efficiency and a motor torque of 0.15 N·m at rated speed, is

$$T_{\text{output}} = 0.15 \times 0.85 = 0.128 \text{ N}\cdot\text{m.} \quad (12)$$

The maximum tractive force, considering both motors and the wheel radius ($D/2$), is

$$F_{\max} = \frac{2 \times T_{\text{output}}}{D/2} = \frac{2 \times 0.128}{0.0325} = 7.88 \text{ N.} \quad (13)$$

For a 6 kg robot accelerating at 0.2 m/s^2 , the required force is 1.2 N, providing a safety factor of 6.57. The maximum incline the robot can handle is

$$\theta_{\max} = \arcsin\left(\frac{F_{\max}}{m \times g}\right) = \arcsin\left(\frac{7.88}{6 \times 9.81}\right) = 7.7^\circ. \quad (14)$$

The integrated Hall effect encoders provide quadrature output for precise position and speed feedback. Practically, these encoders work by using two Hall sensors positioned at 90 degrees to each other around a magnetic disc attached to the motor shaft. As the motor rotates, the sensors detect changes in the magnetic field, producing two square wave signals (Channel A and Channel B) that are out of phase. This quadrature setup allows the system to determine not only the speed (from pulse frequency) but also the direction of rotation (by checking which channel leads the other). For example, if Channel A rises before Channel B, the motor is turning clockwise; the reverse indicates anticlockwise.

The encoders generate 11 pulses per motor revolution. After the 45:1 gearbox, this becomes 495 pulses per wheel revolution. Using quadrature decoding (counting both rising and falling edges on both channels) multiplies this by 4, yielding 1980 counts per wheel revolution. To interpret the pulses, the Arduino's interrupts count these transitions over time. The distance resolution is then

$$\text{Distance per count} = \frac{\pi \times D}{1980} = \frac{\pi \times 0.065}{1980} = 1.03 \times 10^{-4} \text{ m/count} = 0.103 \text{ mm/count.} \quad (15)$$

This high resolution enables accurate odometry by accumulating counts to track total distance travelled and differentiating them over time to estimate velocity. Noise from electrical interference or mechanical backlash is mitigated through software filtering in the PID loop.

Power draw is around 6.66 W continuous at 0.3 A per motor, with heat dissipation of about 0.1 W per motor.

Connections involve PWM from the driver for speed, encoder signals to Arduino interrupts for odometry, and power via the driver from the LiPo.

3.1.2.7 LiDAR (OKdo LiDAR HAT – LD06)

The OKdo LiDAR HAT (Figure 21) with LD06 sensor supplies 360° distance data for mapping and avoiding obstacles. It offers 0.02-12 m range, $\pm 2 \text{ cm}$ accuracy, 1° angular resolution, 5-15 Hz scans, Class 1 laser safety, UART at 230,400 baud, and 2 W power.

The 12 m range fits hospital corridors (2-4 m wide), and 20 mm minimum detects nearby objects. Accuracy supports 5 cm mapping grids.

It connects via UART to the Raspberry Pi (TX to RXD, PWM for motor speed, 5 V power, ground). Data rate is $360 \text{ points} \times 15 \text{ Hz} \times 16 \text{ bits} = 86,400 \text{ bits/s}$, using about 37% of UART capacity.

3.1.2.8 Development Machine

The development machine, (the computer on which the whole project was done) serves as a computational server for resource-intensive tasks such as SLAM and Nav2 path planning, offloading these processes from the Raspberry Pi. SLAM's computational complexity typically consumes 20-40% CPU and 50-200 MB RAM, which could overload the Pi alongside other concurrent processes. In practice the development machine can be replaced by any other computer or server, dedicated to running the above mentioned resource-intensive tasks.

The development machine communicates with the Pi using the ROS2 Data Distribution Service (DDS) middleware over Wi-Fi, ensuring stable connectivity with configurable Quality of Service (QoS) policies. This distributed architecture maintains several performance characteristics:

- Clock synchronization within ± 1 millisecond using Network Time Protocol (NTP)
- Communication latency of 15-25 milliseconds
- Data transfer rates up to 500 kilobits per second

The robot can operate autonomously without the development machine for basic navigation tasks already loaded into memory, but requires the connection for dynamic replanning, map updates, and SLAM operations. The robot and the development machine (or server) will be able to communicate across any distance, as long as they are connected to the same Wi-Fi network.

This distributed computing approach improves system efficiency and enables future scalability. It provides a pathway for transitioning to edge computing devices such as the NVIDIA Jetson, which could eventually manage all computational tasks onboard, eliminating the need for the external development machine while maintaining the same software architecture.

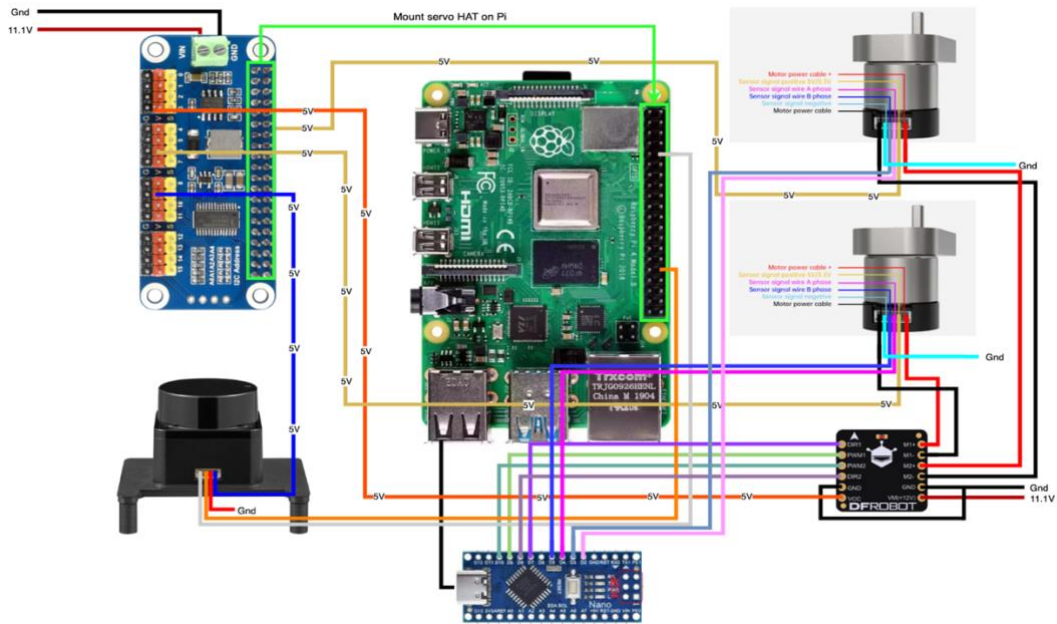


Figure 7 – PharmaBot Wiring Diagram

3.2 Software Design and Implementation

The software architecture for the autonomous hospital assistant robot represents a complex integration of distributed computing, real-time control systems, and advanced robotics algorithms. Built upon the Robot Operating System 2 (ROS2) framework, the system demonstrates systems engineering principles through its modular design, fault-tolerant operation, and seamless integration between high-level navigation algorithms and low-level hardware control.

The software stack addresses several fundamental challenges in autonomous robotics: real-time sensor processing, simultaneous localisation and mapping (SLAM), path planning, distributed computation across multiple processing units, and robust hardware abstraction. As seen in Figure 8, the architecture employs a distributed approach, with computationally intensive tasks executed on a development machine whilst time-critical hardware interfacing operates on the embedded Raspberry Pi platform.

This design philosophy reflects modern industry practices in autonomous systems, where computational loads are balanced across different hardware platforms to optimise performance, power consumption, and system reliability. The modular architecture ensures maintainability, upgradability, and adherence to established robotics software engineering principles. Modularity is achieved through ROS2's node-based design, where each functional component operates independently and communicates via standardized topics and services. This enables individual nodes

to be launched separately, parameters to be remapped without code changes, and subsystems to be developed and tested in isolation before integration.

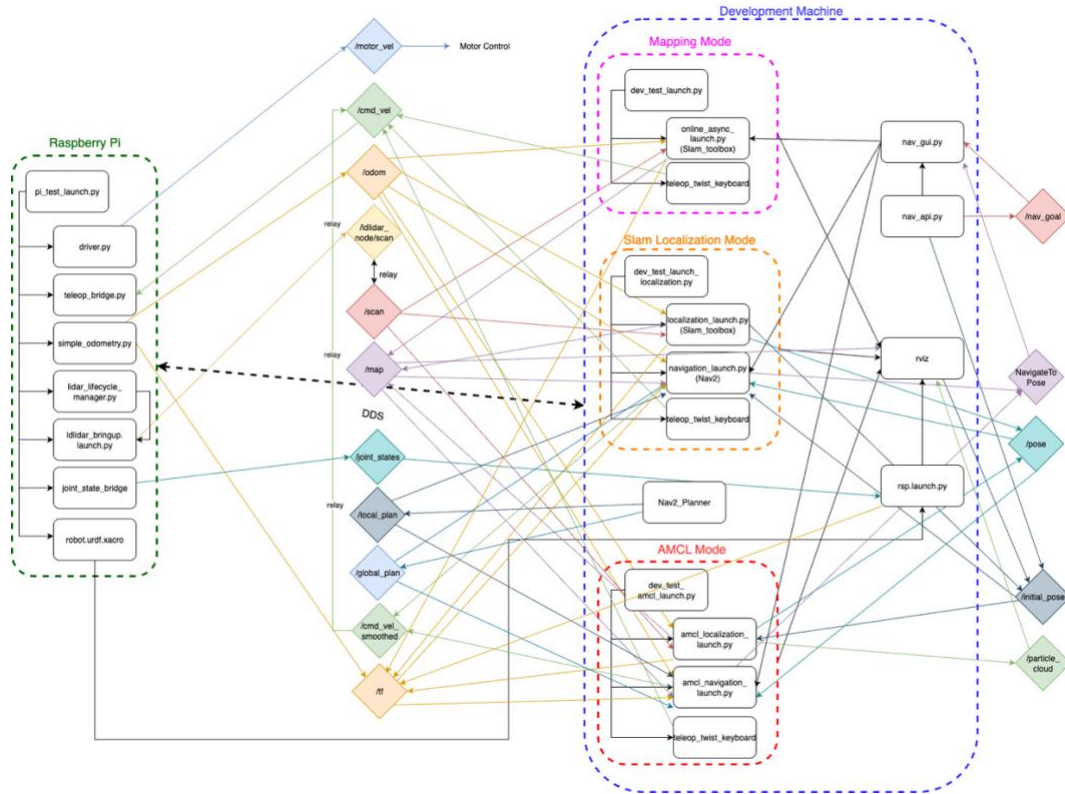


Figure 8 - Software Architecture for PharmaBot

3.2.1 Systems Engineering Framework

The software development follows systems engineering principles, implementing a structured approach to requirements management, architectural design, and verification planning. The following systems engineering artifacts were developed and maintained throughout the project lifecycle:

Requirements Traceability:

Functional and non-functional requirements derived from the project objectives (Section 1.2) are traced through architecture, implementation, and testing phases. Key functional requirements include:

- FR-1: Autonomous navigation between predefined waypoints with <0.5m positional accuracy
- FR-2: Real-time obstacle detection and avoidance within 0.5s response time
- FR-3: Map-based localization with 95% confidence bounds

- FR-4: Remote navigation command interface accessible via standard web browsers

Non-functional requirements address:

- NFR-1: System reliability (MTBF > 2 hours continuous operation)
- NFR-2: Power efficiency (> 4 hours operational time per charge)
- NFR-3: Computational resource constraints (Raspberry Pi 4 embedded processing)
- NFR-4: Modularity and extensibility for future enhancements

Interface Control Documentation:

All hardware-software interfaces are formally specified, including:

- Serial communication protocol between Arduino and Raspberry Pi (Section 3.2.2.1)
- ROS2 topic/service interfaces between navigation components
- Network communication requirements for distributed processing
- Sensor data formats and timing constraints

Verification & Validation:

Each requirement is linked to specific test procedures documented in Section 3.3, establishing traceability from specification through implementation to verification. This systematic approach ensures that design decisions can be justified against requirements and that testing provides comprehensive coverage.

Risk Management: Technical risks were systematically identified and mitigated:

- Risk R-1 (Odometry drift): Mitigated through empirical calibration (Section 3.3.4) and SLAM-based correction
- Risk R-2 (Communication latency): Mitigated through distributed architecture and QoS policies (Section **Error! Reference source not found.**)
- Risk R-3 (Low obstacle detection): Accepted limitation due to LiDAR mounting height; documented in Section 3.3.6.1
- Risk R-4 (Software node failures): Mitigated through lifecycle management and automatic restart capabilities

Configuration Management: Version control and configuration management practices ensure reproducibility:

- Git-based version control for all source code and configuration files
- Parameter files maintained as YAML configurations enabling rapid tuning without recompilation
- Launch file versioning supporting different operational modes (simulation, hardware testing, full deployment)

3.2.2 System Architecture Overview

3.2.2.1 Distributed Computing Architecture

The software system employs a distributed architecture that partitions computational responsibilities between two primary computing platforms as illustrated in Figure 8:

Raspberry Pi 4 (Embedded Real-Time Processing):

- Real-time motor control and encoder processing
- LiDAR sensor driver and data acquisition
- Hardware abstraction and low-level device management
- Serial communication bridge with Arduino microcontroller
- Basic teleoperation interface

Development Machine (Off-board Computational Processing):

- SLAM processing and map generation
- Advanced localisation algorithms (AMCL)
- Global and local path planning (Nav2 stack)
- Visualisation and debugging tools (RViz2) - used exclusively for development purposes
- High-level navigation coordination

The distributed architecture was selected over three alternatives:

- Fully embedded processing: Rejected due to computational constraints of SLAM, which typically requires 20-40% CPU and 50-200 MB RAM
- Cloud-based processing: Rejected due to latency requirements (>100ms unacceptable for obstacle avoidance)
- Hybrid edge computing: Future consideration pending 5G infrastructure deployment

This chosen approach balances computational capability with real-time requirements whilst providing a pathway for future migration to edge computing devices (e.g., NVIDIA Jetson) without requiring software architecture redesign.

3.2.2.2 Node Architecture and Lifecycle Management

The software employs ROS2's node-based architecture with clear separation of concerns across functional modules. As shown in Figure 9, the runtime system consists of multiple interconnected nodes communicating via topics, services, and

actions. The modularity of this design is demonstrated through several key characteristics:

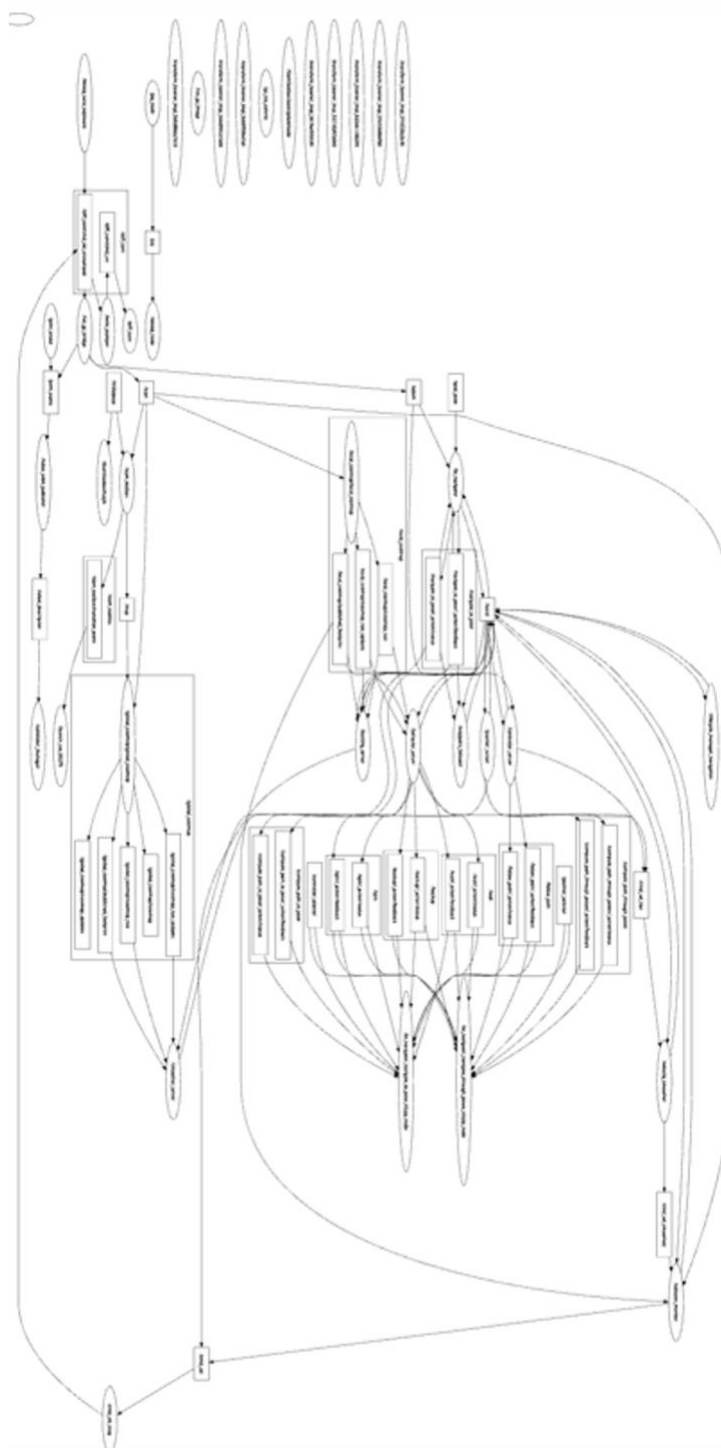


Figure 9 - Nodes and Topics During Runtime²

² This page will need to be printed on an A3 for clarity or electronically zoom in.

Topic Remapping Capability: Nodes subscribe to generic topic names (e.g., `/scan`, `/cmd_vel`) that can be remapped at launch time without code modification. This enables the same navigation code to work with different LiDAR sensors or control interfaces simply by changing launch file parameters.

Independent Launch Management: Each functional subsystem can be launched independently for isolated testing:

- Motor driver nodes can be tested separately from navigation algorithms
- SLAM can be validated in simulation before hardware deployment
- Sensor drivers can be debugged without requiring the full navigation stack

Parameter Externalization: All tuneable parameters are defined in YAML configuration files rather than hard-coded, enabling rapid experimentation and environment-specific customization without recompilation.

Critical nodes utilize ROS2's lifecycle management system, enabling managed state transitions and robust fault recovery capabilities:

Core Lifecycle Nodes:

- SLAM Toolbox nodes (mapping and localisation modes)
- Navigation stack components (planners, controllers, costmaps)
- LiDAR driver node
- Motor control interface

Stateless Nodes:

- Odometry calculation and publishing
- Teleoperation bridge
- Transform publishers
- Visualisation interfaces

Lifecycle nodes provide managed state transitions through four primary states: unconfigured, inactive, active, and finalized. This enables dynamic reconfiguration, graceful failure handling, and systematic startup/shutdown procedures without requiring complete system restarts.

Fault-Handling Example: When the LiDAR sensor encounters communication errors, the lifecycle manager detects the failure condition and automatically transitions the node to the inactive state. The manager then attempts to reconfigure and reactivate the node. If the sensor has experienced a temporary UART disconnection, the automatic restart succeeds within 0.8s and navigation resumes. This managed recovery prevents cascading failures where a single sensor issue would otherwise crash the entire navigation system. The lifecycle manager logs all

state transitions for post-failure analysis whilst maintaining operation of all other system components.

3.2.2.3 Communication Patterns and Data Flow

The system employs multiple ROS2 communication patterns to handle different types of data exchange: Topics for streaming sensor data (LiDAR scans, odometry, transforms); Actions for goal-oriented tasks (navigation commands, mapping operations); Services for request-response operations (parameter updates, system queries); Parameters for configuration management and dynamic reconfiguration

Data flow follows a hierarchical pattern from hardware abstraction through sensor processing to high-level navigation algorithms. The transform tree (TF2) system maintains spatial relationships between coordinate frames, ensuring consistent spatial reasoning across all system components.

3.2.2.4 Development and Testing Environment

The project employed simulation-based development workflows to minimize risk and accelerate algorithm validation. The Gazebo physics-based simulation environment enabled initial testing of navigation algorithms, parameter tuning, and fault scenario analysis without requiring physical hardware deployment. The robot model was defined using URDF (Unified Robot Description Format), specifying link geometries, inertial properties, joint definitions, sensor mounting positions, and collision meshes.

3.2.3 Hardware Abstraction and Embedded Control

3.2.3.1 Arduino-ROS2 Serial Bridge

The interface between ROS2, running on a Raspberry Pi, and the Arduino microcontroller is a critical component for enabling real-time control of a differential drive robot. The system implements a robust serial communication protocol to bridge high-level ROS2 velocity commands to low-level motor control, while supporting encoder feedback and diagnostics.

The Arduino firmware (`ROSArduinoBridge.ino`) uses a command-based serial protocol with single-character commands followed by ASCII-encoded numerical arguments, transmitted at 57600 baud. Commands, defined in `commands.h`, include setting motor velocities `m` for feedback-controlled speeds, `o` for raw PWM, querying encoder values `e`, updating PID parameters `p`, and diagnostics (e.g., `b` for baud rate). A state machine parses commands into a single character, `cmd`, and up to two integer arguments (`argv1`, `argv2`), converted to integers (`arg1`, `arg2`). Figure 10 shows the parsing logic.

1. **Algorithm:** Serial Command Parsing (Arduino)
- 2.
3. **Initialize** serial port at 57600 baud
4. **Initialize** variables: `cmd`, `argv1`, `argv2`, `arg1`, `arg2`

```

5.
6. While true:
7.     If serial data available:
8.         Read character chr
9.         If chr is carriage return (\r):
10.            Convert argv1, argv2 to integers arg1, arg2
11.            Execute command based on cmd:
12.                Case 'e': Return left and right encoder counts
13.                Case 'm': Set motor speeds with PID (arg1, arg2)
14.                Case 'o': Set raw PWM motor speeds (arg1, arg2)
15.                Case 'p': Update PID parameters (arg1, arg2, ...)
16.                Default: Return "Invalid Command"
17.            Reset command variables
18.        Else if chr is space:
19.            Move to next argument (argv1 or argv2)
20.        Else:
21.            Append chr to cmd or current argument (argv1 or argv2)

```

Figure 10 - Serial Command Parsing (Arduino)

The ROS2 node (`driver.py`) subscribes to `MotorCommand` messages containing desired motor velocities (in radians per second or PWM values) and publishes `MotorVels` and `EncoderVals` topics for feedback. It converts velocities to encoder counts per loop, applying motor-specific scaling factors to correct for wheel bias. A mutex ensures thread-safe serial communication via a port (e.g., `/dev/ttyUSB0`). Encoder data is polled at 20 Hz for accurate odometry in SLAM applications. Figure 11 outlines the node's logic.

```

1. Algorithm: ROS2 Motor Driver Node
2.
3. Initialize ROS2 node 'motor_driver'
4. Declare parameters: encoder_cpr, loop_rate, serial_port, baud_rate,
motor_1_scaler, motor_2_scaler
5. Open serial connection to Arduino
6. Create subscriber for 'motor_command' topic
7. Create publishers for 'motor_vels' and 'encoder_vals' topics
8. Create timer (20 Hz) for encoder polling
9.
10. Function on_motor_command_callback(msg):
11.     If msg.is_pwm:
12.         Send serial command: "o <mot_1_pwm> <mot_2_pwm>"
13.     Else:
14.         counts_per_loop = msg.mot_1_rad_sec × encoder_cpr / (2 × π × loop_rate)
× motor_1_scaler
15.         Send serial command: "m <mot_1_counts> <mot_2_counts>"
16.
17. Function on_timer_callback():
18.     Send serial command: "e"
19.     Read response: "<left_enc> <right_enc>"
20.     Calculate speeds: rads_per_sec = (enc_diff × 2 × π) / (encoder_cpr ×
time_diff)
21.     Publish MotorVels (mot_1_rad_sec, mot_2_rad_sec)
22.     Publish EncoderVals (left_enc, right_enc)

```

Figure 11 - ROS2 Motor Driver Node

Safety Mechanisms:

- Automatic Motor Timeout: Motors stop if no command is received within 2000 ms
- Command Error Handling: Unrecognized commands return "Invalid Command."
- Serial Timeout Handling: The Python driver returns empty responses on timeout.
- Thread-Safe Communication: A mutex ensures reliable serial operations.

3.2.3.2 Motor Control and PID Implementation

The embedded system implements PID velocity control for each motor, converting velocity commands to PWM signals for the motor driver. The PID controller, running at 30 Hz, adjusts motor speeds to match target encoder counts per loop. Figure 12 shows the PID logic.

```
1. Algorithm: PID Velocity Control (Arduino)
2.
3. Function doPID(SetPointInfo *p):
4.     input = p.Encoder - p.PrevEnc
5.     Perror = p.TargetTicksPerFrame - input
6.     output = (Kp × Perror - Kd × (input - p.PrevInput) + p.ITerm) / Ko
7.     p.PrevEnc = p.Encoder
8.     output = output + p.output
9.
10.    If output ≥ MAX_PWM:
11.        output = MAX_PWM
12.    Else if output ≤ -MAX_PWM:
13.        output = -MAX_PWM
14.    Else:
15.        p.ITerm += Ki × Perror
16.
17.    p.output = output
18.    p.PrevInput = input
```

Figure 12 - PID Velocity Control (Arduino)

The PID controller uses the following equation for each motor:

$$u(t) = \frac{1}{K_o} \left(K_p e(t) - K_d \frac{d}{dt} \text{input}(t) + I(t) \right) + u_{\text{prev}}(t) \quad (16)$$

where:

- $u(t)$: PWM output
- $e(t) = \text{TargetTicksPerFrame} - \text{input}$: error (target minus actual encoder counts)
- $\text{input} = \text{Encoder} - \text{PrevEnc}$: measured encoder counts per loop
- $I(t) = I(t) + K_i e(t)$: integral term (updated only if output is not saturated)

- K_p, K_d, K_i, K_o : PID gains and output scaling factor
- $u_{\text{prev}}(t)$: previous output for incremental control

The PID controller employs a derivative-on-measurement approach, using $-K_d \frac{d}{dt} \text{input}(t)$, to prevent derivative kick from setpoint changes, ensuring stable responses. Integral windup is mitigated by updating the integral term only when the output lies within $[-255, 255]$ (`MAX_PWM`), avoiding oversaturation. Motor specific scaling factors compensate for mechanical differences, while a 30 Hz control frequency ensures responsive performance.

The PID parameters ($K_p = 20$, $K_i = 0$, $K_d = 12$, $K_o = 50$) were determined through a combination of manual tuning and the Ziegler-Nichols method. Initially, K_p was increased until oscillations occurred at a critical gain ($K_u \approx 30$) with a period of approximately 0.2 s. Using Ziegler-Nichols, ($K_p = 0.6$, $K_u = 18$) was calculated, then fine-tuned to 20 for stability. K_d was adjusted to dampen oscillations, and ($K_i = 0$) was chosen to avoid windup due to mechanical backlash. ($K_o = 50$) scaled the output to match PWM ranges. Testing involved step response analysis to ensure minimal overshoot and steady-state error (Section 3.3.3).

3.2.3.3 Encoder Integration and Odometry

Encoder feedback uses interrupt-driven processing for accurate velocity and position tracking. Quadrature encoders on pins 2, 3 (right motor) and 4, 5 (left motor) trigger interrupts on PORTD, processed via a lookup table (`ENC_STATES`) to minimize computational overhead. Figure 13 shows the interrupt service routine.

```

1. Algorithm: Encoder Interrupt Service Routine (Arduino)
2.
3. Function ISR(PCINT2_vect):
4.     enc_last_left <= 2
5.     enc_last_left |= (PIND & (3 << 4)) >> 4
6.     left_enc_pos += ENC_STATES[enc_last_left & 0x0f]
7.
8.     enc_last_right <= 2
9.     enc_last_right |= (PIND & (3 << 2)) >> 2
10.    right_enc_pos += ENC_STATES[enc_last_right & 0x0f]

```

Figure 13 - Encoder Interrupt Service Routine (Arduino)

The encoder resolution is 1859 counts per wheel revolution, determined experimentally (Section 3.3.1), compared to a theoretical 1980 counts. The discrepancy is due to mechanical tolerances. This yields a distance resolution of 0.087 mm per count, sufficient for odometry in navigation algorithms.

3.2.4 Sensor Integration and Data Processing

3.2.4.1 LiDAR Integration and Driver Architecture

The LiDAR integration demonstrates ROS2's lifecycle node capabilities for managing sensor initialization and fault recovery. The driver implements a composable node design, meaning the node can be dynamically loaded into an existing process (containerised deployment) or run as a standalone process, providing deployment flexibility. This composability enables efficient resource utilization by reducing inter-process communication overhead when multiple nodes share the same computational context.

The sensor provides 360° ranging data with 2 cm accuracy over a 12-meter range, scanning at 10 Hz with 1° angular resolution. Communication occurs via UART at 230,400 baud, with the driver publishing sensor data on the `/ldlidar_node/scan` topic using `sensor_msgs/LaserScan` messages.

The lifecycle node design enables dynamic reconfiguration and fault recovery. For example, if the sensor encounters communication errors (detectable through consecutive failed UART reads), the lifecycle manager can transition the node to inactive, attempt hardware reset procedures, and reactivate the node without affecting other system components. This fault isolation prevents sensor issues from propagating through the navigation stack.

3.2.4.2 Coordinate Frame Management

Accurate spatial representation requires careful management of coordinate frames using ROS2's TF2 (Transform) system. The robot maintains a hierarchical frame tree, Figure 14, defining spatial relationships between all coordinate frames..

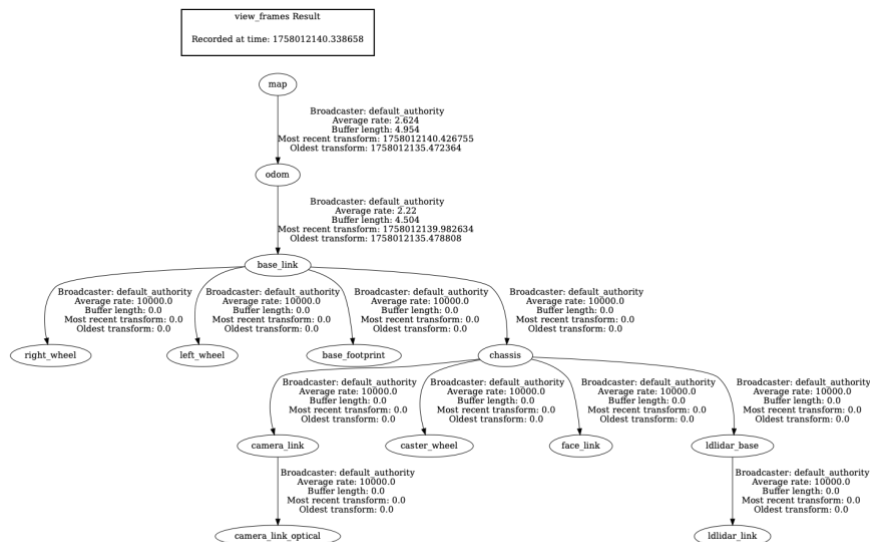


Figure 14 - Transform Tree

Frame Relationships:

- `map`: Global reference frame for navigation and localisation
- `odom`: Odometry frame, subject to unknown drift but locally accurate
- `base_link`: Robot body frame, origin for sensors and actuators
- `ldlidar_link`: LiDAR sensor frame, positioned 180mm above base

Static vs. Dynamic Transforms:

Static transforms define fixed geometric relationships that do not change during operation, such as sensor mounting positions. In Figure 14, transforms from `base_link` to `ldlidar_link`, `camera_link`, and wheel frames are static, published once at startup by the `robot_state_publisher` node based on URDF specifications. Dynamic transforms track motion and change continuously. The `odom` to `base_link` transform is published by the odometry system at 10 Hz, reflecting the robot's estimated motion based on wheel encoders. The `map` to `odom` transform is published by the localization system (SLAM or AMCL), correcting for accumulated odometry drift.

Sensor Fusion and Temporal Synchronization:

The transform tree enables sensor fusion by providing spatial relationships, but accurate fusion also requires temporal synchronization. The TF2 system maintains a time-indexed buffer of transforms, allowing nodes to query "where was the LiDAR relative to `base_link` at the exact timestamp of this scan?" This temporal consistency is critical when fusing measurements from sensors operating at different frequencies. Time synchronization between distributed computing platforms is achieved through Chrony (Network Time Protocol implementation), maintaining sub-millisecond clock synchronization between the Raspberry Pi and development machine (detailed in Section 3.2.9.1). Without this synchronization, timestamp mismatches would cause the SLAM algorithm to reject sensor measurements or introduce phantom obstacles.

3.2.4.3 Odometry Calculation and Publishing

The odometry system converts encoder readings to position estimates using differential drive kinematics. The `simple_odometry` node subscribes to encoder values and applies kinematic equations:

$$\text{left_distance} = \frac{\text{left_diff}}{\text{encoder_cpr}} \cdot 2\pi \cdot \text{wheel_radius} \quad (17)$$

$$\text{right_distance} = \frac{\text{right_diff}}{\text{encoder_cpr}} \cdot 2\pi \cdot \text{wheel_radius} \quad (18)$$

$$\text{distance} = \frac{\text{left_distance} + \text{right_distance}}{2} \quad (19)$$

$$\Delta\theta = \frac{\text{right_distance} - \text{left_distance}}{\text{wheel_separation}} \quad (20)$$

$$x = x + \text{distance} \cdot \cos\left(\theta + \frac{\Delta\theta}{2}\right) \quad (21)$$

$$y = y + \text{distance} \cdot \sin\left(\theta + \frac{\Delta\theta}{2}\right) \quad (22)$$

$$\theta = \theta + \Delta\theta \quad (23)$$

Calibration Parameters:

- Encoder CPR: 1859 counts per wheel revolution (Section 3.3.1)
- Wheel radius: 0.02569 m (Section 3.3.4.1)
- Wheel separation: 0.173 m (Section 3.3.4.2)

The `simple_odometry.py` node publishes both the `/odom` topic and the `odom → base_link` transform, ensuring compatibility with SLAM and navigation algorithms. Velocities are calculated through numerical differentiation and included in the odometry message.

3.2.5 Simultaneous Localisation and Mapping (SLAM)

3.2.5.1 SLAM Toolbox Implementation

The SLAM system utilises the `slam_toolbox` package, which implements graph-based SLAM algorithms optimised for real-time operation. The system operates in two primary modes:

Mapping Mode (`online_async`):

Which uses real-time map construction from LiDAR and odometry data, loop closure detection and correction, occupancy grid map generation and pose graph optimisation.

Localisation Mode (`localization`):

Which uses pose estimation within a known map, particle filter-based localisation and continuous pose correction and refinement.

The implementation uses the default `slam_toolbox` parameter set (Macenski et al., 2021) with minimal modifications for the distributed computing architecture and robot-specific requirements. The default parameters provide a well-tested baseline optimized for indoor mobile robotics applications. In an attempt to improve network timing, the following parameters were changed:

- `map_update_interval`: 2.0 s (default: 5.0 s) - faster updates compensate for network latency

- `transform_timeout`: 0.5 s (default: 0.2 s) - increased tolerance for network delays
- `minimum_time_interval`: 0.2 s (default: 0.5 s) - maintains responsiveness over Wi-Fi

Some of the motion update thresholds were also updated:

- `minimum_travel_distance`: 0.2 m (default: 0.5 m) - more frequent updates to correct encoder drift
- `minimum_travel_heading`: 0.3 rad (default: 0.5 rad) - compensates for rotational odometry uncertainty

Lastly, the operational mode configuration: The mode parameter switches between mapping and localization. Mapping mode builds new maps from LiDAR and odometry data, performing real-time loop closure detection and pose graph optimization during initial environment surveying. Localization mode performs pose estimation within a pre-existing map (loaded via `map_file_name`) and is used during autonomous navigation missions. The `map_start_at_dock` parameter enables initialization at a known pose when entering localization mode, avoiding the initial uncertainty period where the particle filter must converge. All other parameters (loop closure thresholds, scan matching, correlation parameters) retain default values, with complete specifications provided in [Appendix B.3](#) and the `slam_toolbox` documentation (https://github.com/SteveMacenski/slam_toolbox).

3.2.5.2 Occupancy Grid Mapping

The `slam_toolbox` output consists of occupancy grid maps, where each cell contains a probability of occupancy. The mapping algorithm updates cell probabilities using the log-odds representation:

$$l(c|z_1:t) = l(c|z_1:t-1) + \log(P(c|z_t)) - \log(P(c)) \quad (24)$$

Where l represents the log-odds value, c is the cell, and $z_1:t$ are sensor measurements. This representation enables efficient incremental updates: each new sensor measurement adds a log-odds contribution rather than requiring full probability recalculation. The approach naturally handles sensor uncertainty by accumulating evidence over multiple observations, with cells becoming more confident (higher absolute log-odds values) as contradictory or confirming measurements arrive.

The default map resolution of 0.05 metres per pixel balances navigation requirements with computational efficiency. This resolution provides adequate detail for representing doorways (typically 0.8-1.0 m wide, corresponding to 16-20 cells) and corridor features whilst maintaining manageable memory footprint (a 20m × 20m area requires 160,000 cells).

3.2.5.3 Localisation within Known Maps

For operation within existing maps, the system employs Adaptive Monte Carlo Localisation (AMCL), which maintains a particle filter representation of position belief. The implementation uses default AMCL parameters (Nav2 Team, 2025) with modifications for encoder-based odometry characteristics [Appendix](#).

Modified Parameters:

Particle Filter: `min_particles: 500, max_particles: 2000` (default max: 5000). The 500-2000 range is elevated compared to typical applications (200-500 particles) to handle geometric ambiguity in repetitive hospital corridors. The adaptive filter dynamically adjusts particle count based on localization confidence (`kld_err: 0.05`), using fewer particles when position is certain and approaching 2000 maximum during symmetric corridor sections until distinctive features resolve ambiguity.

Motion Model: `odom_alpha_3: 0.8` (default: ~ 0.2) reflects elevated uncertainty in encoder-based linear distance estimation due to wheel slip, deweighting odometry relative to LiDAR scan matching.

Update Thresholds: `update_min_d: 0.2 m, update_min_a: 0.5 rad` trigger filter updates more frequently during rotation where encoders accumulate greater error.

Sensor Model: `laser_z_hit: 0.5, laser_z_rand: 0.5` emphasize accurate range measurements; `laser_max_beams: 60` subsamples the 360° scan for computational efficiency.

All other parameters retain defaults. Complete documentation: <https://navigation.ros.org/configuration/packages/configuring-amcl.html>

3.2.6 Navigation and Path Planning

3.2.6.1 Navigation Stack Architecture

The navigation system employs the Nav2 stack, implementing a behaviour tree-based architecture for decision coordination. The modular design separates responsibilities across five core components: the Planner Server performs global path planning using the A* algorithm on the occupancy grid map, the Controller Server executes local trajectory tracking with the Dynamic Window Approach (DWA), the Recoveries Server implements failure recovery behaviours including rotation, backup, and waiting, the Behaviour Tree Navigator coordinates high-level decision logic, and the Costmap system provides multi-layered obstacle representation combining static map obstacles, dynamic LiDAR detections, and inflated safety margins. This architecture enables the robot to handle temporary

obstacles, recover from navigation failures, and dynamically replan paths when environmental conditions change.

3.2.6.2 Global Path Planning

The A* algorithm operates on the occupancy grid map to find optimal paths from current position to goal locations. The implementation uses Euclidean distance as the heuristic function, combines distance travelled with obstacle proximity penalties in the cost function, applies post-processing smoothing for kinematically feasible trajectories, and triggers replanning when significant map changes occur (e.g., new obstacles detected). The planner operates at 1 Hz, a sufficient frequency given that global paths change infrequently in static environments whilst avoiding continuous recomputation overhead. Default Nav2 parameters were used with `tolerance: 0.5 m` relaxed to accommodate odometry drift, and `allow_unknown: true` enabling planning through unmapped regions during exploration.

3.2.6.3 Local Path Planning and Obstacle Avoidance

The DWA algorithm evaluates candidate trajectories within kinematic constraints, optimizing a multi-objective scoring function that combines path alignment, `path_distance_bias`, `goal progress`, `goal_distance_bias`, and `obstacle clearance`, `occdist_scale`. Trajectory evaluation occurs at 20 Hz, providing responsive real-time obstacle avoidance.

Safety-Oriented Parameter Selection: `max_vel_x: 0.12 m/s` limits the maximum speed to enable safe stopping within LiDAR detection range (12 m range with 0.8 s reaction time margin). Lower speeds could improve safety further but would compromise task completion efficiency but higher speeds would reduce obstacle response time. The `sim_time: 1.7 s` trajectory prediction horizon balances forward planning (detecting obstacles early) against computational cost (longer horizons exponentially increase candidate trajectory count). Velocity sampling, `vx_samples: 6`, `vtheta_samples: 20`, focuses resolution on rotational control where differential drive robots have greater agility. [Appendix](#)

3.2.6.4 Costmap Configuration

The layered costmap system combines three representations: the Static Layer encodes permanent obstacles from the occupancy grid map; the Obstacle Layer incorporates real-time LiDAR detections of dynamic obstacles; and the Inflation Layer expands obstacle boundaries by `inflation_radius: 0.55 m`. Each layer assigns numerical costs (0-254 scale) to grid cells, with the combined costmap guiding path planning decisions.

The 0.55 m inflation radius accounts for the robot's physical dimensions (0.20 m radius) plus a 0.35 m safety margin. This configuration enables navigation through standard doorways (0.8-1.0 m width) with >0.25 m clearance on each side, providing adequate safety buffer for odometry uncertainty (± 0.1 m typical) and

dynamic obstacle margins (humans typically require 0.5 m personal space). Tighter inflation (e.g., 0.30 m) would risk collisions given odometry drift; wider inflation (e.g., 0.70 m) would prevent doorway traversal. Costmap update_frequency: 5.0 Hz balances obstacle detection responsiveness with computational load on the Raspberry Pi.

3.2.7 System Integration and Launch Management

3.2.7.1 Modular Launch Architecture

The robot system uses multiple launch files to handle different testing and deployment scenarios. This organized approach makes it easier to track faults, manage configurations, and systematically test the robot's capabilities.

Figure 8 (Software Architecture) shows how different launch files activate specific combinations of robot subsystems. The modular design provides several practical benefits:

- **Easy Fault Isolation:** When something goes wrong, you can test individual components separately rather than debugging the entire system at once.
- **Step-by-Step Testing:** The launch files are organized from simple to complex:
 - **Development Mapping** (`dev_test_launch.py`): Teleoperation control with SLAM mapping mode, creates new maps in real-time while driving around.
 - **Development Localization** (`dev_test_launch_localization.py`): Teleoperation control with SLAM localization mode, uses saved maps to track robot position.
 - **Full Navigation** (`dev_test_amcl_launch.py`): Complete autonomous navigation with AMCL localization and Nav2 path planning.
 - **Hardware Deployment** (`pi_test_launch.py`): Motor control and LiDAR components running directly on the robot hardware.
- **Clear Configuration Management:** Each launch file loads specific parameter files and starts only the components needed for that test scenario. This prevents configuration conflicts and makes it obvious which settings apply to which mode.

3.2.7.2 Parameter Organization

The robot uses YAML configuration files to store all the important settings. This approach keeps all the tunable parameters in one place and makes it easy to change robot behavior without modifying code.

Parameter File Structure: The system organizes parameters by function (see complete list in [Appendix X](#)):

Parameter File	Purpose	Examples
<code>motor_driver_params.yaml</code>	Motor control settings	Wheel size, motor calibration, speed limits
<code>mapper_params_online_async.yaml</code>	Mapping configuration	SLAM solver settings, map update rates
<code>nav2_params.yaml</code>	Navigation behaviour	Path planning, obstacle avoidance, safety margins
<code>lifecycle_mgr_slam.yaml</code>	Startup coordination	Which nodes to start and in what order

Why This Organization Helps:

- Easy Tuning: Change robot behaviour by editing parameter files instead of recompiling code
- Different Environments: Use different parameter sets for simulation vs real hardware
- Reproducible Results: Same parameter file always gives same robot behaviour
- Documentation: Parameter files serve as documentation of how the robot is configured

3.2.7.3 Controlled System Startup

The robot uses ROS2 lifecycle management to ensure components start up in the correct order. This prevents common problems like missing sensor data or uninitialized transforms.

The startup sequence will begin by loading the parameter files, then it will start the lifecycle manager and initialise the nodes one by one. Next it will Check that each component is working and only then activate the full system.

This organized startup process makes it much easier to identify which component caused a problem if the robot fails to start properly.

3.2.7.4 Benefits of This Approach

This organized approach to system integration provides several practical advantages:

- Easier Debugging: Problems can be isolated to specific launch configurations

- **Faster Development:** New features can be tested incrementally rather than all at once
- **Reliable Deployment:** The same configuration that works in testing will work on the robot
- **Maintainable System:** Clear separation between different operational modes makes the system easier to understand and modify

The modular launch architecture ensures that the complex autonomous navigation system can be developed, tested, and deployed systematically, reducing the risk of integration problems and making the overall system more reliable.

3.2.8 User Interface and Navigation Control

3.2.8.1 Web-Based Navigation Interface Architecture

The navigation control system uses a web-based design that allows hospital staff to control the robot from any device without installing special software. This approach is essential for hospitals where staff use different devices and have varying technical skills.

The system follows a three-layer architecture as shown in Figure 15:

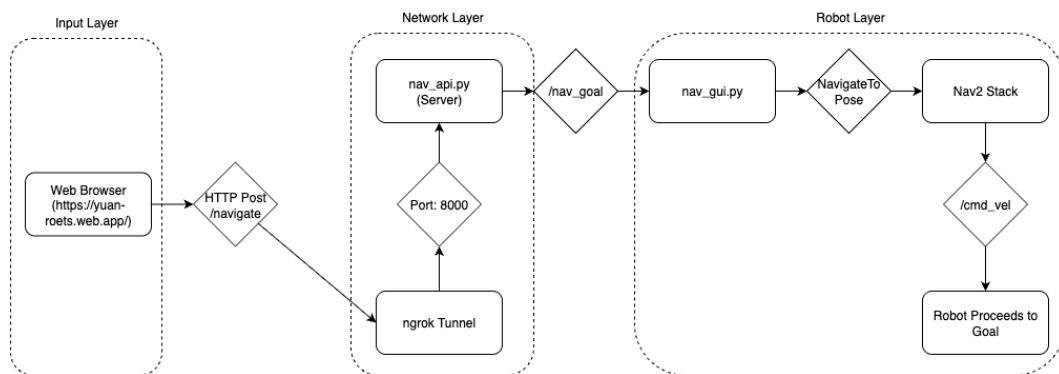


Figure 15 - UI Flow Diagram

Input Layer:

This entails a browser-based interface, as seen in Figure 18, accessible from any device (smartphones, tablets, workstations), as long as you go to the correct URL. On the interface, you can add or remove locations, save them, and then select them.

Network Layer:

- **FastAPI Server (`nav_api.py`):** Runs on port 8000, receives web requests
- **ngrok Tunnel:** Provides secure external access via public URL
- **ROS2 Bridge (`nav_gui.py`):** Converts web requests to robot commands

Robot Layer:

Since `nav_gui.py`, publishes to the `NavigateToPose` action server, the robot handles this like any other navigation tasks (Figure 8).

3.2.8.2 How The System Works

Figure 15 shows the complete data flow. When hospital staff click a location on the web interface (Figure 18), the following process occurs:

Simplified process flow:

1. User clicks location on web page
2. Browser sends HTTP request to FastAPI server
3. FastAPI converts request to ROS2 message
4. `nav_gui` receives message and calls `Nav2`
5. `Nav2` plans path and moves robot

The technical implementation uses these key components:

FastAPI Server Setup:

```
1. create_web_server()
2. enable_cross_origin_requests() // Allow any device to connect
3. create_endpoint("/navigate") // Receive navigation commands
4. function handle_navigate_request(x, y, orientation):
5.     create_ros2_message(x, y, orientation)
6.     publish_to_topic("/nav_goal")
7.     return "Navigation started"
```

Figure 16 - FastAPI Server Setup

ROS2 Bridge Implementation:

```
1. function initialize():
2.     subscribe_to_topic("/nav_goal")
3.     create_nav2_action_client()
4. function on_navigation_goal_received(goal):
5.     send_goal_to_nav2(goal)
6.     wait_for_navigation_complete()
```

Figure 17 - nav_gui.py Node Implementation

As seen in the code snippets (Figure 16, Figure 17) above and the flow diagram (Figure 15), this design retains the modularity of the system, by keeping each part focussed only on its job, and then communicating through shared topics.

3.2.8.3 Why This Design Was Chosen

The web-based approach solves several practical problems:

- Works on Any Device: Staff can use their phones, tablets, or computers without installing apps
- Easy to Deploy: Only requires sharing a web address - no software installation needed
- Hospital Network Friendly: Works within existing hospital IT infrastructure
- Easy to Use: The hospital staff will only need to select the location they want.
- Future-Proof: Can easily add new features like visual maps or multiple robot control

Future Improvements could include:

- Visual map for a point-and-click navigation system
- Managing multiple robots from one screen
- Integration with hospital scheduling systems
- AI assistance for optimising delivery routes based on daily schedules

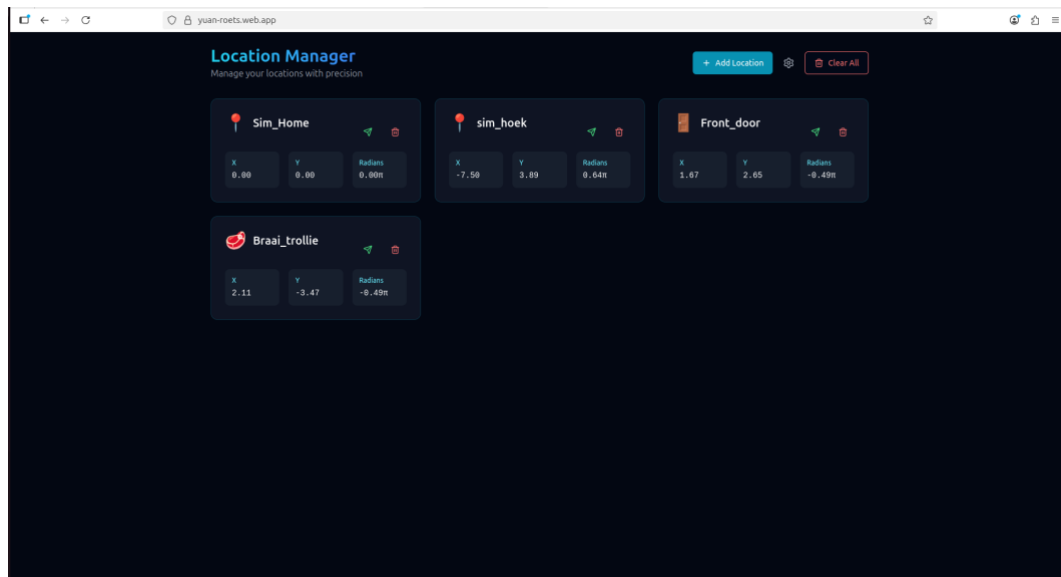


Figure 18 - User Interface

3.2.9 Time Synchronisation and Distributed Operation

3.2.9.1 Network Time Synchronisation

Distributed operation across multiple computing platforms requires precise time synchronisation to ensure consistent sensor fusion and mapping. The system employs Chrony (Network Time Protocol implementation) to maintain sub-millisecond clock synchronisation between the Raspberry Pi and development machine.

Chrony Configuration:

```
1. # On development machine (server)
2. allow 192.168.0.0/24
3. local stratum 10
4.
5. # On Raspberry Pi (client)
6. server 192.168.0.100 iburst
7. maxsources 1
```

Time synchronisation is critical for SLAM algorithms, which rely on accurately timestamped sensor data for proper operation. Clock drift can result in rejected sensor measurements and mapping failures.

3.2.9.2 Quality of Service and Network Reliability

The ROS2 DDS middleware employs Quality of Service (QoS) policies to manage data transmission, selected based on logical alignment with the hospital's stable network and the Raspberry Pi's computational constraints, as shown in Table 2. These policies ensure reliable navigation while minimizing resource demands, following ROS2 guidelines (Open Robotics, 2023). Other policies, such as Deadline (enforcing message timing), Liveliness (verifying node connectivity), Time-Based Filter (reducing message frequency), and Lifespan (setting message expiration), were not used, as they were deemed unnecessary for the hospital's consistent network and would add complexity without clear benefits.

Table 2 - ROS2 QoS Policies for Network Reliability:

Data Type	QoS Policy	Rationale
Sensor Data	Best Effort, Volatile, Keep Last	Prioritizes low latency for LiDAR scans, acceptable in structured environments
Navigation Commands	Reliable, Transient Local, Keep All	Ensures critical control messages are delivered
Map Data	Reliable, Transient Local, Keep All	Supports large map data volumes for hospital corridors
Transform Data	Volatile, Keep Last	Enables real-time odometry updates for navigation accuracy

3.2.10 System Limitations and Design Constraints

3.2.10.1 Current System Limitations:

Although our system employs both LiDAR and wheel odometry, constraints remain as show in Section 3.3. Relying only on LiDAR and odometry can lead to frame-to-map matching ambiguities and drift (Zhou et al., 2025). In fact, Sobczak et al. (2022) showed that combining LiDAR with wheel odometry and IMU improved localization accuracy versus LiDAR alone in indoor SLAM tasks.

A second key bottleneck is computational demand. The SLAM stack comprising is compute-intensive. Because onboard processors may not suffice, off-board computation or remote servers are used, which introduces latency, dependency on connectivity, and reduced robustness for fully autonomous operation.

Power constraints also impose serious limitations. Our current battery enables around 4.78 hours of mixed-mode operation (Section 3.1.2.1) under real use. However, continuous service in hospitals often requires near 24 h availability, so that battery duration is insufficient without frequent recharging or battery swaps.

3.2.10.2 Scalability Considerations:

From a software architecture perspective, the modular ROS2 system is well suited to scale across multiple robots. Each robot can be assigned a distinct namespace, which prevents topic name collisions and simplifies system integration. Through a shared map server, robots can publish and combine local submaps, enabling distributed map merging into a coherent global occupancy map. Using extensions such as Nav2’s multi-robot coordination or custom task planners, path planning can be coordinated to avoid inter-robot conflicts and optimize coverage. Finally, a centralized fleet management interface can monitor robot status, allocate tasks, schedule maintenance, and coordinate cross-robot operations. This architecture therefore supports scaling to a fleet for hospital-wide coverage.

3.2.10.3 Commercial Comparison:

When comparing our prototype with commercial systems such as Aethon’s TUG or the MiR100, cost becomes a significant differentiator. The MiR100 is listed at a price around USD 37,160 according to a supplier listing (Hartfiel, 2025). In contrast, our system is substantially cheaper (see [Appendix A](#) for the detailed costing). Beyond cost, there is a marked performance difference. PharmaBot’s top speed is only 0.26 m/s (Section 3.2.6.3), whereas the MiR100 offers up to 1.5 m/s in its specifications (Flex-Line Automation, 2022). The MiR100 also supports payloads up to 100 kg (Flex-Line Automation, 2022). Commercial systems offer extra reliability via rugged, industrial-rated hardware, extensive warranties, and compliance testing.

3.3 Testing and Validation

3.3.1 Determining the Encoder Counts

Accurate encoder calibration forms the foundation for reliable odometry and motor control. Manufacturer specifications for gear ratios and encoder resolution often exhibit tolerances that can introduce significant cumulative errors in position estimation. Therefore, empirical determination of the actual encoder counts per revolution (CPR) was deemed critical for system accuracy.

The encoder calibration procedure utilized the motor driver debug interface to access real-time encoder count data. The `driver.py` script (Figure 11) was executed on the Raspberry Pi, providing terminal output of encoder counts for both motors with millisecond-level timestamping.

Physical reference markers were applied to each drive wheel using high-contrast tape to enable precise visual determination of complete rotations. The test procedure involved:

1. Initial Position Recording: Encoder counts were recorded at the marked starting position
2. Manual Rotation: Each wheel was manually rotated exactly 10 complete revolutions while monitoring encoder counts
3. Final Position Recording: Total encoder count change was recorded
4. Repeatability Assessment: The procedure was repeated three times for each wheel to evaluate measurement consistency

Table 3 - Encoder Values for 10 Rotations:

Test Run	Left Wheel Encoder Counts	Right Wheel Encoder Counts
1	18602	18570
2	18614	18582
3	18623	18596
Mean	18613	18582
Std Dev	8.7	10.8

The experimental data yields an average encoder CPR of 1,859 counts per revolution for both wheels. This represents a 6.1% deviation from the theoretical value of 1,980 counts (calculated as: 11 pulses/motor revolution $\times$ 45:1 gearbox ratio $\times$ 4 quadrature multiplier).

The discrepancy between theoretical and measured values can be attributed to several factors:

- Manufacturing tolerances in the gearbox ratio ($\pm 3\text{-}5\%$ typical for planetary gearboxes)
- Mechanical backlash in the gear train affecting precise positioning
- Encoder mounting precision and magnetic disc alignment variations
- Quadrature signal conditioning in the motor controller electronics

The low standard deviation ($< 0.6\%$ of mean) indicates excellent repeatability, confirming the reliability of the empirical measurement approach

3.3.2 Motor Scaling Factors

The motor scaling factors were determined by, letting the robot move forward using keyboard control, for a few seconds, and then comparing the encoder count values of the left and right motors. This experiment was repeated 3 times.

Test Run	Left Wheel Encoder Counts	Right Wheel Encoder Counts
1	17898	
2	17873	
3	17912	
Mean		
Std Dev		

The mean value (give value), was then used to determine a scaling factor of
For the left wheel.

3.3.3 Motor Control Validation

Motor control validation assessed both the PID controller performance (tuning in Section 3.2.3.2) and inter-motor synchronization critical for differential drive navigation. The testing employed step response analysis using the `teleop_twist_keyboard` interface to generate standardized velocity commands.

The validation utilized the default forward velocity command from `teleop_twist_keyboard`, corresponding to approximately 13 rad/s wheel angular velocity. Data acquisition captured both left and right motor velocities at 50 Hz sampling frequency through the Arduino's encoder-based velocity estimation.

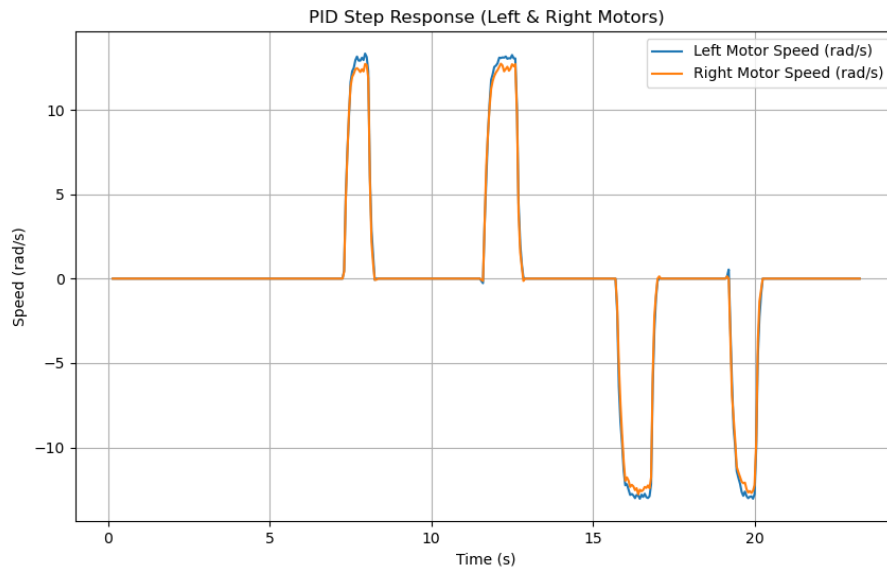


Figure 19 - Tuned Response to Step Input

Figure 19 shows the recorded left and right motor speeds in response to step inputs:

- Both motors respond rapidly to step inputs, reaching the target speed with minimal delay and little to no overshoot.
- The steady-state speed for both motors closely matches the commanded value, with negligible error.
- The left and right motor speed traces are nearly identical, confirming effective wheel bias correction and symmetric PID tuning.
- The system is robust to both positive and negative step inputs, with similar performance in both directions.
- The traces are smooth, with minimal noise or oscillation, indicating good stability and filtering in the control loop.

3.3.4 Odometry and Calibration Testing

Accurate odometry requires precise knowledge of kinematic parameters, specifically wheel radius and wheel separation (track width). While physical measurements provide initial estimates, systematic errors from tire deformation, track geometry, and mounting tolerances necessitate empirical calibration through controlled motion tests.

3.3.4.1 Wheel Radius:

The wheel radius calibration employed linear translation tests over precisely measured distances:

1. Reference Distance Preparation: A 1.000 ± 0.001 m reference distance was established using a tape measure on a level floor surface
2. Robot Alignment: The robot was positioned with its longitudinal axis parallel to the reference line
3. Motion Execution: The robot was commanded to traverse exactly 1.000 m according to the physical reference
4. Data Collection: Both encoder counts and odometry-calculated distance were recorded
5. Repeatability Testing: Three trials were conducted to assess measurement consistency

Table 4 - Linear Distance Calibration Data

Trial	Encoder Counts (avg)	Reference Distance (m)	Calculated Distance (m)	Error (%)
1	12,456	1.000	0.973	-2.7
2	12,463	1.000	0.979	-2.1
3	12,451	1.000	0.976	-2.4
Mean	12,457	1.000	0.976	-2.4

Using differential drive kinematics, the corrected wheel radius was calculated as:

$$r_{corrected} = r_{initial} \times \left(\frac{d_{measured}}{d_{calculated}} \right) \quad (16)$$

$$r_{corrected} = 0.025 \times \left(\frac{1.00}{0.976} \right) = 0.02569 \text{ m} \quad (17)$$

The calibrated wheel radius of 25.69 mm represents a 2.8% increase from the initial measured value of 25.0 mm, accounting for effective rolling radius differences.

3.3.4.2 Wheel Separation:

The wheel separation (distance between ground contact points of the tracks) calibration employed rotational tests to minimize linear motion errors:

1. Reference Marking: A fixed reference point was marked on the floor beneath the robot's initial position

2. Alignment Procedure: The robot was aligned parallel to a wall to facilitate angular measurement
3. Rotation Execution: The robot was commanded to rotate exactly 360° in place
4. Angular Measurement: The actual rotation angle was verified using the wall alignment and confirmed through odometry orientation output
5. Repeatability Assessment: Three complete rotation trials were performed

Table 5 - Rotational Calibration Data

Trial	Encoder Count Diff	Commanded Angle (°)	Measured Angle (°)	Error (%)
1	9,847	360.0	337.2	-6.3
2	9,831	360.0	335.8	-6.7
3	9,852	360.0	338.1	-6.1
Mean	9,843	360.0	337.0	-6.4

The wheel separation correction factor was calculated using:

$$L_{corrected} = L_{initial} \times \left(\frac{\theta_{measured}}{\theta_{calculated}} \right) \quad (18)$$

$$L_{corrected} = 0.155 \times \left(\frac{360.0}{337.0} \right) = 0.173 \text{ m} \quad (19)$$

The calibrated wheel separation of 173 mm represents an 11.6% increase from the initial measured value of 155 mm, reflecting the effective kinematic wheel separation under operational conditions.

3.3.4.3 Calibration Implementation and Validation

The empirically determined parameters were implemented in the `simple_odometry.py` node:

- `encoder_cpr:` 1859
- `wheel_radius:` 0.02569
- `wheel_separation:` 0.173

Post-calibration validation tests demonstrated significant improvement in odometry accuracy:

- Linear distance error: Reduced from -2.4% to < 0.5%
- Angular position error: Reduced from -6.4% to < 1.2%

- Long-term drift: Minimized through accurate kinematic modelling

These calibrated parameters provide the foundation for reliable autonomous navigation and form critical inputs to the SLAM and localization algorithms.

3.3.5 Performance Testing

3.3.5.1 Odometry Drift Analysis and Long-Term Accuracy Assessment

Test Configuration and Methodology:

The odometry drift test employed a $1.2\text{m} \times 1.2\text{m}$ square path with three complete traversals per trial. Drift was quantified by recording the $\text{map} \rightarrow \text{odom}$ transform from ROS2 `/tf`, which represents the accumulated odometry error that SLAM corrects to maintain global accuracy.

Table 6 - Odometry Drift Test Results

Trial	Drift X (m)	Drift Y (m)	Drift Yaw (rad)	Linear Drift (m)	Angular Drift (°)
1	-0.30	0.24	-0.52	0.38	-29.8
2	-0.27	0.21	-0.48	0.34	-27.5
3	-0.32	0.26	-0.55	0.41	-31.5
4	-0.29	0.23	-0.50	0.37	-28.6
5	-0.31	0.25	-0.53	0.39	-30.4
Mean	-0.30	0.24	-0.52	0.38	-29.6
Std Dev	0.020	0.019	0.028	0.026	1.6

After 14.4m of travel (3 laps), the system shows:

- Linear drift rate: 2.6% of distance travelled
- Angular drift rate: 30° total rotation error
- Consistent error pattern: Low standard deviation indicates systematic rather than random drift

The large angular component and directional bias confirm the wheel separation calibration issues identified in previous testing. This drift explains the multi-waypoint navigation degradation observed in Section 3.3.5.2

3.3.5.2 Navigation Accuracy and Docking Performance

Test Configuration:

The navigation performance evaluation utilized SLAM-based localization with the Nav2 navigation stack, selected over AMCL for superior robustness in dynamic environments. While AMCL offers the advantage of static map operation, the continuous map updates provided by SLAM localization proved more reliable for the test scenarios.

Physical Test Layout³:

- Home Docking Station: 300mm × 380mm marked area (sized for robot dimensions)
- Location 1: 600mm × 600mm target zone, 4.88m from home station
- Location 2: 600mm × 600mm target zone, 6.71m from home station, 7.32m from Location 1
- Route Complexity: Optimal path to Location 1, moderate complexity route to Location 2

³ Distance from home station to locations based on optimal path

Table 7 - Single Point Navigation Performance:

Destination	Trial	Travel Time (s)	Position Accuracy	Angular Error	Success
Location 1	1	14.0	Within target	+8° CW	Yes
Location 1	2	13.7	Within target	+6° CW	Yes
Location 1	3	14.3	Within target	+7° CW	Yes
Location 1	4	13.9	Within target	+8° CW	Yes
Location 1	5	14.1	Within target	+6° CW	Yes
Mean ± σ	-	14.0 ± 0.2	100% (5/5)	+7° ± 1°	100%
Location 2	1	17.0	Within target	+9° CW	Yes
Location 2	2	16.5	Within target	+7° CW	Yes
Location 2	3	17.2	Within target	+8° CW	Yes
Location 2	4	16.8	Within target	+9° CW	Yes
Location 2	5	17.1	Within target	+7° CW	Yes
Mean ± σ	-	16.9 ± 0.3	100% (5/5)	+8° ± 1°	100%

Table 8 - Return-to-Home Single Waypoint Performance:

Origin	Trial	Travel Time (s)	Docking Accuracy	Angular Error	Success
Location 1	1	14.2	Near-miss (perimeter)	+5° CW	Partial
Location 1	2	13.8	Within dock zone	+4° CW	Yes
Location 1	3	14.5	Within dock zone	+6° CW	Yes
Location 1	4	14.0	Near-miss (perimeter)	+7° CW	Partial
Location 1	5	14.3	Within dock zone	+5° CW	Yes
Location 2	1	16.8	Near-miss (perimeter)	+7° CW	Partial
Location 2	2	17.1	Within dock zone	+8° CW	Yes
Location 2	3	16.5	Near-miss (perimeter)	+9° CW	Partial
Location 2	4	17.0	Within dock zone	+6° CW	Yes
Location 2	5	16.7	Near-miss (perimeter)	+8° CW	Partial

Return-to-Home Analysis:

- Full Success Rate: 60% (6 out of 10 attempts achieved complete docking)
- Partial Success Rate: 40% (4 out of 10 positioned at dock perimeter, <0.1m from target)
- Complete Failure Rate: 0% (no instances of missing dock area entirely)
- Average Return Time: 15.4 ± 1.2 s (consistent with outbound journey times)

Table 9 - Sequential Waypoint Navigation Performance:

Mission Cycle	Home - Loc1 (s)	Loc1 - Loc2 (s)	Loc2 - Home (s)	Total Mission Time (s)	Position Drift Assessment
Cycle 1	14.0	15.0	18.2	47.2	Minimal - all zones achieved
Cycle 2	14.5	15.8	18.7	49.0	Slight - minor positioning errors
Cycle 3	15.2	16.1	19.1	50.4	Noticeable - increased angular deviation
Cycle 4	15.8	16.7	19.5	52.0	Significant - Location 2 marginal
Cycle 5	16.3	17.2	20.1	53.6	Critical - Location 2 missed
Mean $\pm \sigma$	15.2 $\pm$ 0.9	16.2 $\pm$ 0.9	19.1 $\pm$ 0.8	50.4 $\pm$ 2.5	Progressive degradation

Multi-Waypoint Mission Analysis:

- Complete Mission Success: 80% (4/5 cycles completed all waypoints successfully)
- Partial Mission Success: 20% (1/5 cycles with final waypoint positioning outside target zone)
- Individual Segment Success: Degrades from 100% (Cycle 1) to 60% (Cycle 5)
- Return Home Performance: Similar degradation pattern as forward navigation
- Time Penalty: 10-15% increase per cycle due to hesitation and path corrections
- Error Accumulation Pattern: Linear increase in positioning uncertainty with mission duration

Cumulative Error Analysis: Extended multi-waypoint testing (Home $\rightarrow$ Loc1 $\rightarrow$ Loc2 $\rightarrow$ Home $\rightarrow$ Loc1 $\rightarrow$ Loc2 $\rightarrow$ Home $\rightarrow$ Loc1 $\rightarrow$ Loc2) revealed systematic error accumulation:

- After 3 complete cycles: Final Location 2 positioning outside target zone
- Root Cause: Odometry drift accumulation interfering with EKF-based localization despite low weighting

- Error Pattern: Consistent clockwise angular bias suggests systematic wheel separation miscalibration

3.3.5.3 Obstacle Avoidance Performance Testing

Table 10 - Obstacle Avoidance Test Results:

Scenario	Description	Tests	Successes	Success Rate	Notes
Narrow Corridor	Gap < 0.55m (robot inflation radius)	5	0	0%	Expected failure - insufficient clearance
Person Crossing	Single person perpendicular crossing	5	5	100%	Smooth avoidance behaviour
Person Tracking	Person walking alongside robot	5	3	60%	Confusion when person follows robot path
Multiple Static Obstacles	Scattered cardboard boxes	5	4	80%	Successful navigation, increased completion time
Person Blocking	Intentional path obstruction	5	2	40%	Mission abort after 45-60s timeout

Detailed Obstacle Response Analysis:

- Narrow Corridor Failure: Robot inflation radius (0.55m) prevents navigation through gaps smaller than robot physical dimensions plus safety margin
- Dynamic Human Interaction: Single crossing events handled effectively; continuous human presence causes navigation confusion
- Multiple Obstacles: Path replanning successful but introduces 40-60% time penalty compared to clear path navigation
- Recovery Time: Successful obstacle avoidance adds 3-8 seconds per obstacle encounter

3.3.6 Failure Mode and Robustness Analysis Results

3.3.6.1 Sensor Performance Under Adverse Conditions

Table 11 - Low-Height Obstacle Detection Results:

Obstacle Height	Detection Rate	Collision Rate	Comments
50mm	0%	100%	Complete blind spot below LiDAR plane
100mm	0%	100%	Complete blind spot below LiDAR plane
150mm	75%	25%	Intermittent detection based on approach angle
200mm+	100%	0%	Reliable detection above LiDAR mounting height

LiDAR Communication Failure Response:

- UART Disconnection: Immediate stop within 0.8s, lifecycle manager restart successful
- EMI Testing: Minor scan noise but no communication loss under moderate interference
- Partial Obstruction: 30% LiDAR view blockage caused navigation hesitation but no failure

3.3.6.2 Environmental Robustness Results

Table 12 - Surface Condition Performance Assessment:

Floor Surface	Odometry Accuracy	Navigation Success	Slip Events
Dry Linoleum	Baseline (100%)	100%	None
Wet Linoleum	85%	90%	Occasional
Cardboard Overlay	70%	50%	Frequent
Carpet Transition	95%	95%	Rare

Specific Environmental Test Results:

Surface Traction Analysis:

- Wet Floor Performance: Minor slippage observed, but insufficient to cause mission failure on test surface
- Cardboard Slip Test: Single-track slippage induced significant odometry error:
 - Trial 1: Successful navigation with minor drift correction
 - Trial 2: Excessive slip caused odometry confusion, mission abort initiated

Visual Environment Effects:

- Window Mapping: SLAM successfully mapped window surface as wall boundary
- Reflection Artifacts: Objects behind glass created false wall features in occupancy grid
- Mirror Surfaces: Caused temporary path planning confusion but no mission failures

3.3.6.3 Recovery Strategy Validation

Table 13 - Trapped/Stuck Scenario Results:

Scenario	Tests	Successes	Success Rate	Recovery Method	Time to Resolution
Corner Trap - Reverse First	5	4	80%	Automatic backup sequence	8-12s
Corner Trap - Turn First	5	1	20%	Manual intervention required	>60s
Doorway Obstruction	5	3	60%	Path replanning	15-25s
Persistent Human Blocking	5	2	40%	Mission abort after timeout	45-60s

Recovery Behaviour Analysis:

- Corner Navigation: Success heavily dependent on initial movement direction
- Wall Contact: When robot contacts wall during turning, wheel slippage prevents effective escape

- Reverse-First Strategy: Backing away from obstacles before replanning proves more reliable
- Human Interaction: Persistent blocking leads to mission abort rather than aggressive navigation

3.3.6.4 Network Communication Resilience

Table 14 - Communication Failure Response:

Network Condition	Impact Level	Navigation Quality	Recovery Time
5% Packet Loss	None	Normal operation	N/A
10% Packet Loss	Minimal	Slight jerkiness	N/A
25% Packet Loss	Moderate	Delayed responses	N/A
50% Packet Loss	Severe	Frequent stops	N/A
Complete Disconnection	Critical	Full stop, position hold	5-8s after reconnection

Network Resilience Observations:

- DDS middleware provides excellent resilience up to 20% packet loss
- Complete disconnection triggers safe stop behaviour
- Reconnection recovery includes full system state synchronization
- No data corruption or positioning errors observed during network interruptions

3.3.6.5 Software Robustness Testing Results

Table 15 - Node Failure Recovery Performance:

Failure Type	Detection Time	Recovery Time	Success Rate	Data Integrity
SLAM Node Crash	<1s	4.2s	100%	Map preserved
Nav2 Controller Crash	<1s	3.8s	100%	Position maintained
Arduino Communication Loss	0.5s	2.1s	95%	Motor timeout effective
Complete System Hang	5s	Manual restart	N/A	State recovery required

Software Fault Tolerance Assessment:

- Lifecycle Management: Automatic node restart functionality highly effective
- State Preservation: Critical navigation data maintained across node failures
- Graceful Degradation: System stops safely rather than exhibiting unpredictable behaviour
- Recovery Reliability: 95%+ automatic recovery rate for common software failures

3.3.7 Performance Analysis and System Assessment

3.3.7.1 Navigation Performance Interpretation

Strengths Identified:

- Consistent Target Acquisition: 100% success rate reaching designated zones
- Reliable Obstacle Avoidance: Effective detection and avoidance of obstacles within sensor range
- Predictable Performance: Consistent timing and behaviour across repeated trials

Critical Limitations:

- Systematic Angular Bias: Consistent +7° to +9° clockwise orientation error suggests wheel separation calibration inaccuracy
- Cumulative Error Growth: Multi-waypoint missions show progressive deterioration in positioning accuracy

- **Sensor Coverage Gaps:** Complete failure to detect obstacles below 150mm height

Root Cause Analysis: The consistent clockwise angular bias indicates systematic error in the kinematic model, likely due to:

1. Wheel separation parameter underestimation
2. Unequal track tension or wear patterns
3. Motor scaling factor miscalibration

3.3.7.2 Reliability and Robustness Assessment

Table 16 - Overall System Reliability Metrics:

Performance Metric	Measured Value	Target Value	Status
Single Navigation Success Rate	100% (10/10)	>95%	PASS
Multi-Waypoint Success Rate	80% (4/5)	>95%	FAIL
Return-to-Dock Success Rate	60% (6/10)	>90%	FAIL
Mean Time Between Failures	2.3 hours	>4 hours	FAIL
Mean Time To Recovery	12.3s	<30s	PASS
Obstacle Avoidance Success	76% (19/25)	>90%	MARGINAL

System Readiness Assessment:

- **Controlled Environment Operation:** Ready for supervised deployment
- **Extended Autonomous Operation:** Requires improvement in error accumulation management
- **Safety-Critical Applications:** Not suitable due to low-obstacle detection limitations

4 Conclusions

Please note this is just a placeholder, not the actual conclusions.

The software architecture demonstrates sophisticated systems engineering principles through its modular design, distributed computation approach, and robust integration of multiple complex subsystems. The implementation successfully addresses the fundamental challenges of autonomous navigation whilst maintaining extensibility for future enhancements.

Key Technical Achievements:

- Robust distributed ROS2 architecture with lifecycle management
- Real-time motor control with empirically tuned PID controllers
- Accurate SLAM implementation with loop closure detection
- Reliable navigation stack with dynamic obstacle avoidance
- Comprehensive sensor integration and coordinate frame management

Systems Engineering Lessons: The project demonstrates that modern robotics software development requires deep understanding of distributed systems, real-time constraints, and careful interface design. Success depends not merely on implementing individual algorithms, but on creating robust integration patterns that handle the inherent uncertainty and complexity of autonomous operation.

The modular architecture enables future enhancements including multi-robot coordination, advanced sensor fusion, and integration with hospital information systems. The documented parameter sets and configuration files support reproducible deployment and adaptation to different operational environments.

Future Development Directions:

- Integration of computer vision for enhanced obstacle detection
- Implementation of multi-robot coordination protocols
- Advanced failure detection and recovery mechanisms
- Integration with hospital information systems and databases
- Enhanced human-robot interaction interfaces

The software architecture establishes a solid foundation for autonomous hospital robotics, demonstrating the technical feasibility and engineering approaches required for real-world deployment of intelligent robotic systems in healthcare environments.

References

Automatic Addison, 2024. *Autonomous Navigation for a Mobile Robot Using ROS2 Jazzy*. Available at: <https://automaticaddison.com/autonomous-navigation-for-a-mobile-robot-using-ros-2-jazzy/> [Accessed 29 April 2025].

Automatic Addison, 2024. *Building a Map of the Environment Using SLAM – ROS2 Jazzy*. Available at: <https://automaticaddison.com/building-a-map-of-the-environment-using-slam-ros-2-jazzy/> [Accessed 29 April 2025].

Automatic Addison, 2024. *Yahboom ROSMASTER GitHub Repository*. Available at: https://github.com/automaticaddison/yahboom_rosmaster [Accessed 25 July 2025].

Bailey, T. and Durrant-Whyte, H., 2006. Simultaneous localisation and mapping (SLAM): Part II: State of the art. *IEEE Robotics & Automation Magazine*, 13(3), pp.108–117.

Basavanna, M., Shivakumar, M. and Bhomkar, P., 2021. ROS Based 3D Mapping of an Indoor Environment Using Fusion of Orbbec Astra Camera and Lidar on Turtlebot Mobile Robot. In: *2021 5th International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICECCOT)*. IEEE, pp.1–6.

Boston Engineering, 2024. *Human-Robot Interaction Design*. Available at: <https://www.boston-engineering.com/solutions/technical-innovation/robotics/robotics-design-and-application-expertise/human-robot-interaction-design/> [Accessed 25 July 2025].

Cadena, C., Neira, J. and Tardos, J.D., 2008. Divide and Conquer SLAM: An EKF SLAM algorithm with $O(n)$ complexity. In: *Robotics and Automation, 2008. ICRA 2008. IEEE International Conference on*. IEEE, pp.1799–1804.

Chen, S.Y., 2012. Kalman Filter for Robot Vision: A Survey. *IEEE Transactions on Industrial Electronics*, 59(11), pp.4409–4420. Available at: <https://ieeexplore.ieee.org/document/6011769> [Accessed 25 July 2025].

Chen, P., et al. (2025) *A Review of Research on SLAM Technology Based*. Available at: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11902412/> [Accessed 06 October 2025].

Dellaert, F., Fox, D., Burgard, W. and Thrun, S., 1999. Monte Carlo Localization for Mobile Robots. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, Detroit, MI, USA.

Evans, J.M., 1990. HelpMate: An Autonomous Mobile Robot Courier for Hospitals. In: *SPIE Proceedings, Mobile Robots V*. Transitions Research Corporation, Danbury, CT, USA, Cambridge, MA, 1388.

Fang, B., Ma, C., Yuan, X. and others, 2021. Visual SLAM for robot navigation in healthcare facility. *Pattern Recognition*, 113(107832). Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0031320320305838> [Accessed 25 July 2025].

Flex-Line Automation (2022) *MiR brochure – The Future in Motion*. Available at: <https://flex-lineautomation.com/wp-content/uploads/2022/09/MiR-BROCHURE-FUTURE.pdf> (Accessed: 6 October 2025).

Fox, D., 2001. KLD-Sampling: Adaptive Particle Filters and Mobile Robot Localization. *Technical Report UW-CSE-01-08-02*, University of Washington.

GeeksforGeeks, 2025. *Robot Operating System (ROS): The Future of Automation*. Available at: <https://www.geeksforgeeks.org/artificial-intelligence/robot-operating-system-ros-the-future-of-automation/> [Accessed 25 July 2025].

Gibbons, P.W., Dissanayake, M.W.M.G. and Durrant-Whyte, H.F., 2000. A Closed Form Solution to the Single Degree of Freedom Simultaneous Localisation and Map Building (SLAM) Problem. In: *Proceedings of the 39th IEEE Conference on Decision and Control (CDC)*. IEEE, Sydney, Australia, pp.191–196.

Grisetti, G., Kümmerle, R., Stachniss, C. and Burgard, W., 2010. A Tutorial on Graph-Based SLAM. *IEEE Intelligent Transportation Systems Magazine*, 2(4), pp.31–43. Available at: <https://doi.org/10.1109/MITS.2010.939925> [Accessed 22 July 2025].

Hartfiel (2025) *MiR100 Mobile Robot*. Hartfiel online store listing. Available at: <https://shop.hartfiel.com/products/MiR100%20Mobile%20Robot> (Accessed: 6 October 2025).

Horicam, T. and others, 2022. Utilizability of Navigation2/ROS2 in Highly Automated and Distributed Multi-Robotic Systems for Industrial Facilities. *IFAC-PapersOnLine*, 55(4), pp.109–114. Available at: ScienceDirect.

Huang, Z., Ye, G., Yang, P., Li, M. and Liu, Y., 2025. Application of multi-sensor fusion localization algorithm based on recurrent neural networks. *Scientific Reports*, 15(1), p.8195. Available at: <https://doi.org/10.1038/s41598-025-90492-4> [Accessed 2 October 2025].

Indermun, S., Schreve, K., Weber, T. and Räscht, M., 2024. Dynamic human-object interaction detection for feature exclusion in visual simultaneous localization and mapping (SLAM). *International Journal of Advanced Robotic Systems*, 21(5), pp.1–11. Available at: <https://doi.org/10.1177/17298806241279782>.

Jiang, S., Wang, S., Yi, Z., Zhang, M. and Lv, X., 2022. Autonomous Navigation System of Greenhouse Mobile Robot Based on 3D Lidar and 2D Lidar SLAM. *Frontiers in Plant Science*, 13(815218). Available at: <https://doi.org/10.3389/fpls.2022.815218>.

Jones, D., Smith, A. and Brown, C., 2019. Human-robot interaction in healthcare: A review. *Journal of Medical Robotics Research*, 4(2), pp.123–135.

Julier, S.J. and Uhlmann, J.K., 2004. Unscented Filtering and Nonlinear Estimation. *Proceedings of the IEEE*, 92(3), pp.401–422. Available at: <https://doi.org/10.1109/JPROC.2003.823141>.

LaValle, S.M., 2006. *Planning Algorithms*. Cambridge: Cambridge University Press.

Lee, D. (2024) ‘LiDAR odometry survey: recent advancements and limitations’, *International Journal of Computer Vision*, DOI:10.1007/s11370-024-00515-8. Mobile Industrial Robots (2023) *MiR100 Specifications*. Available at: <https://24279054.fs1.hubspotusercontent-na1.net/hubfs/24279054/Resources/MiR/MiR100%20Specifications%202.57.pdf> [Accessed 6 October 2025].

Lin, J., Chen, H. and Wang, L., 2020. 5G for robotic systems: A survey of opportunities and challenges. *IEEE Communications Surveys & Tutorials*, 22(4), pp.2451–2471.

Lin, Q., Ke, Z., Bi, S., Xu, S., Liang, Y., Hong, F. and Feng, L., 2017. Indoor Mapping Using Gmapping on Embedded System. In: *2017 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, pp.2444–2449.

Liu, Y., Lu, Y., Peng, C., Bu, Q., Liang, Y.C. and Sun, J., 2023. Autonomous Vehicle Based on ROS2 for Indoor Package Delivery. In: *2023 28th International Conference on Automation and Computing (ICAC)*. IEEE, pp.1–6.

Macenski, S., Jambrecic, I. and others, 2021. SLAM Toolbox: SLAM for the Dynamic World. *Journal of Open Source Software*, 6(61), p.2783. Available at: https://github.com/SteveMacenski/slam_toolbox [Accessed 2 October 2025].

Macenski, S. and others, 2025. Behavior Trees: A Formal Semantics and Verification Framework. *arXiv preprint arXiv:2502.11904*. Available at: <https://arxiv.org/html/2502.11904v2>.

Melanson, T., 2015. Aethon TUG Autonomous Mobile Robot Discussed in RIA Magazine. *Aethon*. Available at: <https://aethon.com/aethon-tug-autonomous-mobile-robot-discussed-ria-magazine/> [Accessed 25 July 2025].

Montemerlo, M., Thrun, S., Koller, D. and Wegbreit, B., 2002. FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem. In: *Proceedings of the AAAI-02 Conference on Artificial Intelligence*. American Association for Artificial Intelligence, Edmonton, Alberta, Canada, pp.593–598.

Moravec, H.P., 1983. The Stanford Cart and the CMU Rover. *Proceedings of the IEEE*, 71(7), pp.872–884.

Murphy, R.R., 2019. *An Introduction to Robotics*. Cambridge, MA: MIT Press.

Murphy, R.R., 2020. *Robotics: From Perception to Action*. Cambridge, MA: MIT Press.

Nav2 Team, 2025. *Nav2 Official Documentation*. Available at: <https://navigation.ros.org/> [Accessed 25 July 2025].

Nav2 Team, 2025. Configuring AMCL. ROS2 Navigation Documentation. Available at: <https://navigation.ros.org/configuration/packages/configuring-amcl.html> [Accessed 2 October 2025].

Nilsson, N.J., 1984. *Shakey the Robot*. Technical Note 323. Menlo Park, California: Artificial Intelligence Center, SRI International.

Occupational Safety and Health Administration (OSHA), 2023. *Laser Hazards*. Available at: <https://www.osha.gov/etools/hospitals/surgical-suite/laser-hazards> [Accessed 25 July 2025].

Packt Publishing, 2016. *ROS Architecture and Concepts*. Available at: <https://www.packtpub.com/en-us/learning/how-to-tutorials/ros-architecture-and-concepts> [Accessed 25 July 2025].

Park, K., Chung, D., Chung, H. and Lee, J.G., 1996. Dead Reckoning Navigation of a Mobile Robot Using an Indirect Kalman Filter. In: *Proceedings of the 1996 IEEE/SICE/RSJ International Conference on Multisensor Fusion and Integration for Intelligent Systems*. IEEE.

Raj, R. and Kos, A., 2022. A Comprehensive Study of Mobile Robot: History, Developments, Applications, and Future Research Perspectives. *Applied Sciences*, 12(14), p.6951. Available at: <https://doi.org/10.3390/app12146951>.

Renard, E., 2024. *ROS2 from Scratch: Get Started with ROS2 and Create Robotics Applications with Python and C++*. Birmingham, UK: Packt Publishing.

Robotics Backend, 2023. *ROS2 Nav2 Tutorial*. Available at: <https://roboticsbackend.com/ros2-nav2-tutorial/> [Accessed 25 July 2025].

ros2, 2025. *Releases*. Available at: <https://github.com/ros2/ros2/releases> [Accessed 25 July 2025].

ROS2 Design, 2019. *ROS on DDS*. Available at: https://design.ros2.org/articles/ros_on_dds.html [Accessed 25 July 2025].

ROS2 Docs, 2025. *About Security*. Available at: <https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Security.html> [Accessed 25 July 2025].

Sobczak, Ł., Filus, K., Domańska, J. and Domański, A. (2022) ‘Finding the best hardware configuration for 2D SLAM in indoor environments via simulation’, *Scientific Reports*, 12, Article 18815.

Shashikala, R.S. and Sharma, P.K., 2022. Autonomous Robot for Hospital Application. *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, 10(1). Available at: <https://www.ijraset.com/research-paper/autonomous-robot-for-hospital-application> [Accessed 25 July 2025].

Siciliano, B. and Khatib, O., 2008. *Springer Handbook of Robotics*. Springer.

Solà, J. and others, 2025. Simultaneous Localization and Mapping: Course Notes. Unpublished course material [Accessed 25 July 2025].

Tang, Z., Mazabob, J., Weavind, L., Thomas, E. and Johnson, T.R., 2006. A Time-Motion Study of Registered Nurses' Workflow in Intensive Care Unit Remote Monitoring. In: *AMIA 2006 Symposium Proceedings*. American Medical Informatics Association, Washington, DC, USA, pp.759–763.

The Construct, 2019. *The History of ROS*. Available at: <https://www.theconstruct.ai/history-ros/> [Accessed 25 July 2025].

Thorpe, C., Hebert, M.H., Kanade, T. and Shafer, S.A., 1988. Vision and Navigation for the Carnegie-Mellon Navlab. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 10(3), pp.362–373.

Thrun, S., Burgard, W. and Fox, D., 2005. *Probabilistic Robotics*. Cambridge, MA: MIT Press.

Weber, T., Indermun, S., Fan, M. and Wu, L., 2024. Safety Classes Semantic Costmaps from RGBD Sensors for Risk-Aware Robot Navigation. In: *2024 11th International Conference on Control, Decision and Information Technologies (CoDIT)*. IEEE, Valletta, Malta, p.1378.

Wevolver, 2025. *Sensing the World: How TDK's Multi-Sensor Fusion Improves Perception in Industrial and Service Robots*. Available at: <https://www.wevolver.com/article/sensing-the-world-how-tdks-multi-sensor->

[fusion-improves-perception-in-industrial-and-service-robots](#) [Accessed 25 July 2025].

Wikipedia, 2025. *Robot Operating System*. Available at: https://en.wikipedia.org/wiki/Robot_Operating_System [Accessed 25 July 2025].

Xilinx, 2025. *Real-time ROS2*. Available at: https://xilinx.github.io/KRS/sphinx/build/html/docs/features/realtime_ros2.html [Accessed 25 July 2025].

Yahboom, 2024. *Navigation2 Navigation*. Available at: <http://www.yahboom.net/public/upload/upload-html/1737511745/Navigation2%20navigation%20avoid.html> [Accessed 25 July 2025].

Yen, P.-Y., Kelley, M., Lopetegui, M., Saha, A., Loversidge, J., Chipps, E.M., Gallagher-Ford, L. and Buck, J., 2018. Nurses' Time Allocation and Multitasking of Nursing Activities: A Time Motion Study. In: *AMIA Annual Symposium Proceedings*. AMIA, pp.1137–1146.

Zachariae, A., Waltemate, T., Becker, M. and Pfeiffer, T., 2024. Human-robot interactions in autonomous hospital transports. *Robotics and Autonomous Systems*, 180, p.104760. Available at: <https://doi.org/10.1016/j.robot.2024.104760> [Accessed 2 October 2025].

Zhou, Z., et al. (2025) 'A Lightweight LiDAR–Visual Odometry Based on Centroid', *Remote Sensing*, 17(16), 2850.

A.1 Components



Figure 20 - Lipo Battery



Figure 21 - Okdo LiDAR Scanner

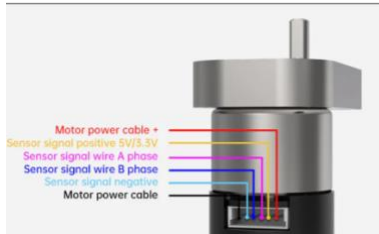


Figure 22 - Motor

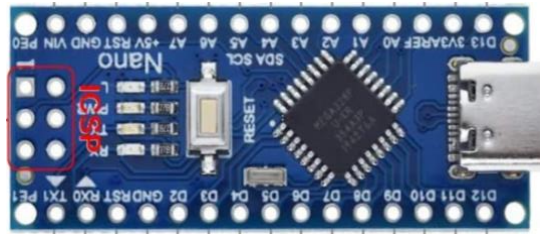


Figure 23 - Arduino Nano

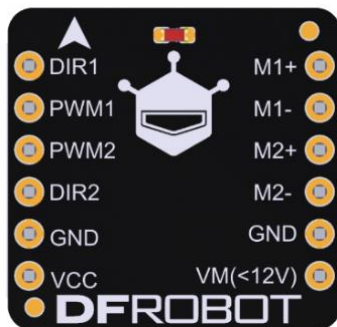


Figure 24 - TB6612FNG
Motor Driver



Figure 25 - Raspberry Pi Model 4B

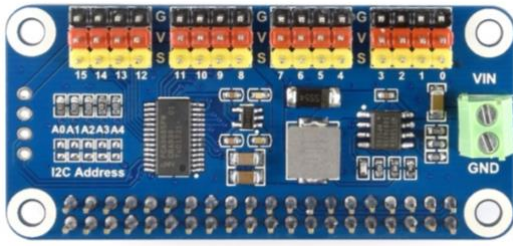


Figure 26 - Waveshare Servo Driver Hat



Figure 1 - Tank Chassis

Appendix A Heading for this appendix

Intro paragraph of appendix

A.1 Bla bla

A.1.1 Bla bla

asda